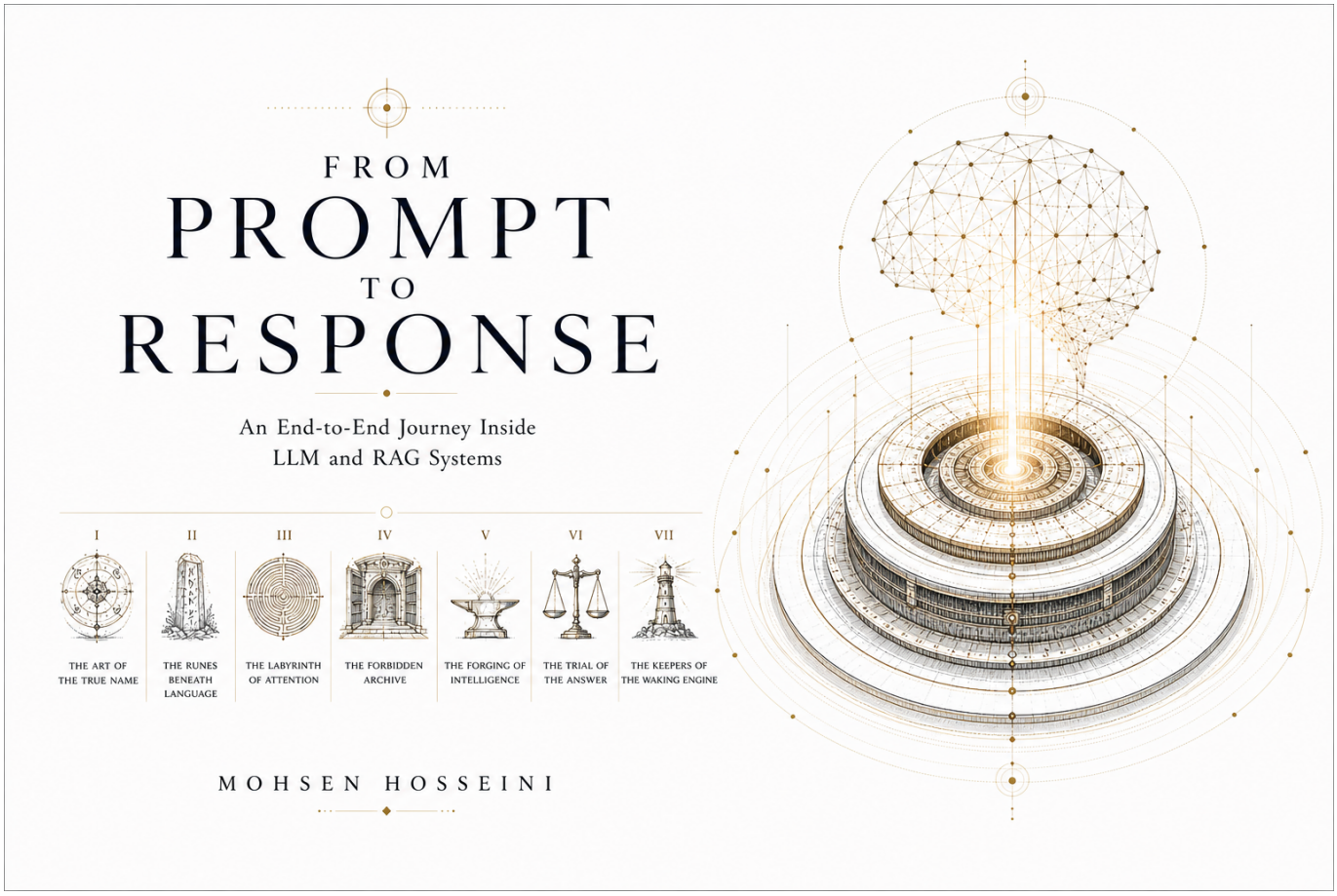
From Prompt to Response

An End-to-End Journey Inside LLM and RAG Systems

Table of contents

Welcome



A technical deep-dive into the full LLM stack, from training and alignment through serving, retrieval, evaluation, and production operation. Every chapter is built around a single question that unlocks a layer of the system.

Designed for engineers and researchers who work with LLM systems and want to understand not just how to use them, but how they behave end-to-end under real constraints.

Part I

The Art of the True Name

Chapter 1

the intro

Part II

The Runes Beneath Language

Chapter 2

APIs and Model Serving

The canonical question for this chapter: *A trained model is a file on a disk. How does it become something millions of people can query simultaneously, with low latency, high reliability, and predictable cost?*

Figure 2.1: The journey through the Model Mind.

The prompt has been written and structured. Now it needs to travel somewhere. This chapter covers everything between the user pressing Enter and the model receiving the request: the wire format, the serving infrastructure, and the engineering decisions that make large-scale LLM deployment economical.

2.1 The gap between training and deployment

Training produces a checkpoint: a collection of weight tensors saved to disk. At this point, the model can do nothing on its own. It has no interface. It cannot receive requests. It does not know what a user is.

Turning that checkpoint into a production service requires solving a completely different set of engineering problems from those that were involved in training. During training, we try to optimize for producing as many tokens as possible per second. On the other hand, during serving, the optimization focuses on latency, concurrency, reliability, and cost, which means we want to respond to requests quickly, serve many users simultaneously, remain available under failure, and do all of this economically.

These requirements require a different approach. The same GPU cluster that runs training efficiently is not automatically an efficient serving infrastructure. Serving is its own engineering discipline, with its own abstractions, its own failure modes, and its own optimization techniques.

This chapter covers the full stack from trained checkpoint to the API endpoint a user or application calls.

2.2 The anatomy of an LLM API

The standard interface for production LLM services is an HTTP API, almost universally following the conventions established by OpenAI's API.

2.2.1 The chat completions endpoint

The primary endpoint for most production use:

`POST /v1/chat/completions`

Request body:

```
{
  "model": "gpt-4o",
  "messages": [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "What is the capital of France?"}
  ],
  "max_tokens": 100,
  "temperature": 0.7,
  "stream": false
}
```

Response:

```
{
  "id": "chatcmpl-abc123",
  "object": "chat.completion",
  "created": 1699000000,
  "model": "gpt-4o",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "The capital of France is Paris."
      },
      "finish_reason": "stop"
    }
  ],
  "usage": {
    "prompt_tokens": 26,
    "completion_tokens": 9,
    "total_tokens": 35
  }
}
```

Several design decisions in this interface iluminates the realities of LLM serving:

Messages array instead of a single string. Conversations have a clear structure consisting of a system prompt, user turns, and assistant turns. The API explicitly defines the structure, instead of relying on concatenated strings and expecting the model to interpret the formatting correctly.

Token counts in the response. Because LLM cost and latency are proportional to token count, the API reports exactly how many tokens were consumed allowing costs to be monitored and controlled programmatically.

`finish_reason`. Why did the model stop generating? `stop` means it produced an end-of-sequence token. `length`

means it reached the `max_tokens`. `content_filter` means the response was filtered. These signals help to determine whether the response is complete

`stream` parameter. Whether to stream tokens as they are generated or return the complete response when done. Streaming is architecturally significant and covered in detail in chapter "REFF".

2.2.2 The completions endpoint (legacy)

An older interface that takes the raw string prompt rather than the structured messages array:

```
{
  "model": "gpt-3.5-turbo-instruct",
  "prompt": "The capital of France is",
  "max_tokens": 10
}
```

This endpoint represents an earlier era of LLM deployment, when models were not chat tuned and the interface was closer to a text continuation service. It is still useful for non-chat use cases but has been superseded by chat completions for most applications.

2.2.3 Key parameters

`temperature` controls the randomness of sampling. At temperature 0, the model always selects the most likely next token (greedy decoding). At temperature 1, it samples from a full distribution. Above 1, the distribution flattens, producing more random and unstable outputs. Temperature interacts with the decoding strategy covered in chapter "REFF".

`top_p` (nucleus sampling) this is an alternative to temperature for controlling randomness. The model samples from the smallest set of tokens in which the cumulative probability exceeds `top_p`. Setting `top_p = 0.9` restricts sampling to tokens that together account for 90% of the probability mass that means the long tail of unlikely tokens is ignored.

`max_tokens` sets the maximum number of tokens to generate. It functions as a cost and latency cap. If the model reaches this limit before producing a stop token, `finish_reason` is `length` and the response may be end mid-sentence.

Stop sequences are one or more strings that cause halt in generation immediately if the model produces them. They are useful for structured generation where you know the output should end at a specific delimiter.

`frequency_penalty` and `presence_penalty` discourage repetition. `frequency_penalty` applies a penalty proportional to how many time a token has already appeared. `presence_penalty` applies a flat penalty to any token that has appeared at all. Both help the model to prevent from getting stuck in repetitive loops.

`n` specifies how many independent completions to generate for a single prompt. Useful for best-of-n sampling where you generate multiple responses and select the best one.

`logprobs` returns the log probabilities of the selected tokens and also the top-k most likely tokens at each position. It is used for calibration, analysis, and classification tasks where you care more about the model's confidence, not just its output.

2.3 From checkpoint to serving: the loading pipeline

Before a model can serve requests, it must be loaded. For a model with hundreds of billions of parameters, this is a nontrivial task.

2.3.1 Checkpoint format and sharding

Training checkpoints are often sharded due to the fact that the full model does not fit in the memory of a single storage node or loading process. A 70B parameter model in BF16 requires approximately 140 GB of storage and Loading it

into a GPU memory requires careful orchestration.

Common checkpoint formats:

- **PyTorch `.pt` / `.bin`**: the standard PyTorch format which can be slow to load due to pickle deserialization
- **SafeTensors**: a faster and safer alternative which can be loaded significantly faster via memory-mapped files
- **GGUF**: a format used by llama.cpp, optimized for inference on consumer hardware and it is able to quantize weights natively

2.3.2 Quantization for serving

Training typically utilizes BF16 weights (2 bytes per parameter). For serving, quantization reduces this further:

INT8 quantization stores weights as 8-bit integers (1 byte per parameter). This achieves a 2× memory reduction which causes a small quality loss that is mostly negligible for generation tasks.

INT4 quantization uses 4-bit integers (0.5 bytes per parameter) for a 4× memory reduction. Causes more quality loss and needs careful calibration. Enables running 70B models on consumer GPUs that could not otherwise fit them.

GPTQ is a post-training quantization technique that calibrates 4-bit quantization using a small dataset, minimizing quality loss from reduced precision.

AWQ (Activation-aware Weight Quantization) quantizes weights while accounting for the distribution of activations, producing better results than naive quantization at the same bit width.

The tradeoff is consistent: the quantization reduces memory requirements while increase throughput, at the cost of some output quality. For production serving where cost matters, INT8 is nearly universal. INT4 is used when memory is severely constrained or when we want to serve on consumer hardware.

2.3.3 Tensor parallelism for large models

A 405B parameter model in BF16 requires approximately 810 GB of GPU memory that is far beyond a single GPU's capacity. In these scenarios, loading requires distributing the model across multiple GPUs using tensor parallelism (splitting individual weight matrices across GPUs) or pipeline parallelism (assigning different layers to different GPUs).

The same parallelism strategies used in training apply to serving, but with different constraints: training prioritizes throughput (large batches, high utilization), while serving prioritizes latency (fast first-token generation, smaller batches).

2.4 The serving architecture

2.4.1 The inference server

The core component: a process that holds the model weights in GPU memory and handles requests. The major implementations:

vLLM is the dominant open-source inference server. It introduces PagedAttention (covered in detail in chapter "REFF") for efficient KV cache management, and supports tensor parallelism, continuous batching, and most major model architectures.

TensorRT-LLM (NVIDIA) is NVIDIA's inference optimization library. It applies kernel fusion, INT8/FP8 quantization, and hardware-specific optimizations. Typically the fastest option on NVIDIA hardware but requires compilation for specific model shapes which is a trade between flexibility and performance.

llama.cpp is a pure C++ implementation of LLaMA-family models, optimized for CPU and consumer GPU inference. Not competitive with vLLM for high-throughput serving, but it is unmatched for accessibility and local deployment.

2.4.2 The API gateway

In front of the inference server sits the API layer:

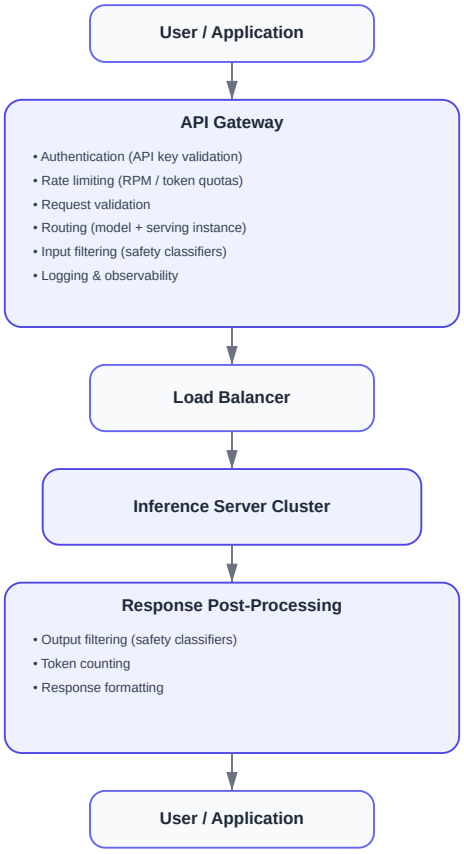


Figure 2.2: The API gateway.

The API gateway is not a performance bottleneck what it does is to process the metadata. It is critical for security (authentication prevents unauthorized access), cost control (rate limiting prevents abuse), reliability (routing distributes traffic and handles failures), and observability (logging every request enables debugging and billing).

2.5 Batching: the key to serving efficiently

A single inference server instance can handle many requests simultaneously. How it batches those requests together affects throughput and GPU utilization.

2.5.1 Naive static batching

The simplest approach is to wait until you have `batch_size` requests, process them together, return all results.

The problem is that LLM generation is variable length. On request might generate 10 tokens while the other might generate 500. In a static batch, all requests must wait for the longest one to finish. GPU utilization drops as shorter requests complete and their slots sit idle, waiting.

2.5.2 Continuous batching

The key innovation for efficient LLM serving, introduced by [1] and popularized by vLLM.

Instead of batching at the request level, continuous batching operates at the iteration level meaning it happens at every token generation step. After each token is generated:

1. Check which sequences in the current batch have finished
2. Remove finished sequences from the batch
3. Add new incoming requests to fill the freed slots
4. Run the next generation step on the updated batch

```
Step 1: [A , B , C ]         ← three requests, first token
Step 2: [A , B , C ]
Step 3: [A , B , C_stop]     ← C finishes after 3 tokens
Step 3: [A , B , D ]         ← D immediately fills C's slot
Step 4: [A , B , D ]
Step 5: [A_stop, B , D ]     ← A finishes
Step 5: [E , B , D ]         ← E fills A's slot
```

The GPU is always processing a full batch. None of the slots sit idle waiting for the longest sequence to finish. On real workloads, this approach can improve throughput by 2 to 3 times compared to a naive static batching. Continuous batching requires careful KV cache management which explains why PagedAttention exists.

2.5.3 Prefill and decode phases

LLM generation has two distinct phases with different compute profiles:

Prefill: processing the input prompt. All prompt tokens are processed in parallel. This phase is compute bound and the bottleneck is GPU arithmetic throughput.

Decode: generating output tokens one at a time. Each step processes one single new token, attending to all previous tokens via the KV cache. This phase is memory-bandwidth-bound and the bottleneck is reading the model weights and KV cache from GPU memory.

Because prefill and decode have different bottlenecks, mixing them in the same batch is not optimal. **Chunked prefill** and **disaggregated serving** are emerging techniques that separate prefill and decode across different hardware or batch slots to maximize utilization of each phase independently.

2.6 KV cache management

The KV cache stores the key and value tensors computed for each previous token, avoiding recomputation on every generation step. It is the central memory management challenge in serving, and will be explained in chapter “REFF”.

A brief introduction here, because it directly constrains serving architecture:

For a model with `L` layers, `h` KV heads, `d_k` key/value dimension, and a sequence of length `n`:

```
KV cache size = 2 × L × h × d_k × n × bytes_per_element
```

For LLaMA 2 70B (L=80, h=8 GQA heads, d_k=128) in BF16:

```
Per token: 2 × 80 × 8 × 128 × 2 = 327,680 bytes  320 KB
4,096 tokens: 320 KB × 4,096  1.3 GB
32,768 tokens: 320 KB × 32,768  10.5 GB
```

At high concurrency with long contexts, KV cache competes directly with model weights for GPU memory.

2.6.1 PagedAttention

The key innovation in vLLM, inspired by virtual memory paging in operating systems.

Traditional KV cache reserves a contiguous memory block for each sequence, sized to accommodate the maximum possible length. This causes internal fragmentation (reserved memory that goes unused when sequences are shorter than expected) and external fragmentation (free memory that cannot be used because it is not contiguous).

PagedAttention allocates KV cache in fixed-size pages of tokens, with a page table mapping logical sequence positions to physical memory locations. Memory is allocated and freed one page at a time as sequences grow and complete:

```
Sequence A: [Page 3, Page 7, Page 12]     ← pages need not be contiguous
Sequence B: [Page 1, Page 9]
Sequence C: [Page 2, Page 4, Page 5, Page 11]
Free pages: [Page 6, Page 8, Page 10, ...]
```

This will eliminates fragmentation and enables an almost perfect memory utilization. It also enables prefix sharing which means if two requests share a common system prompt, the KV cache for that prefix can be shared across both sequences without duplication.

2.7 Latency and throughput tradeoffs

Serving performance is characterized by two metrics that pull against each other:

Time to first token (TTFT) is how long the user waits before seeing any output. This is determined primarily by prefill time and since longer prompts take longer to prefill it is the metric users perceive most directly.

Time between tokens (TBT) is how fast tokens arrive after the first one. This is determined by decode speed. For a 70B model on a single A100, 20 to 60 tokens per second is typically expected.

Throughput is total tokens processed per second across all concurrent requests. Maximized by large batch sizes, which directly conflict with low TTFT.

Production systems manage this tradeoff through maximum batch size limits (cap batch size to maintain acceptable latency), priority queuing (prioritize interactive requests over batch workloads), and SLA-based scheduling (different latency targets for different user tiers).

2.8 Key takeaways

- A trained model checkpoint becomes a production service through a pipeline involving quantization, multi-GPU loading, an inference server, and an API gateway, since training and serving are distinct engineering disciplines
- The chat completions API format, including the messages array, key parameters, and token usage reporting, reflects the practical realities of LLM serving rather than arbitrary design choices
- Continuous batching operates at the token level rather than the request level, keeping GPUs fully utilized despite variable-length generation and serving as the main efficiency improvement for high-throughput systems
- KV cache memory is the central serving constraint because it grows with sequence length and batch size and competes directly with model weights for GPU memory, while PagedAttention manages it using virtual memory paging
- Prefill is compute bound and decode is memory bandwidth bound, so they have fundamentally different performance profiles and can be optimized independently
- TTFT and throughput trade off against each other, and the appropriate balance depends on whether the system serves interactive applications or batch workloads

---

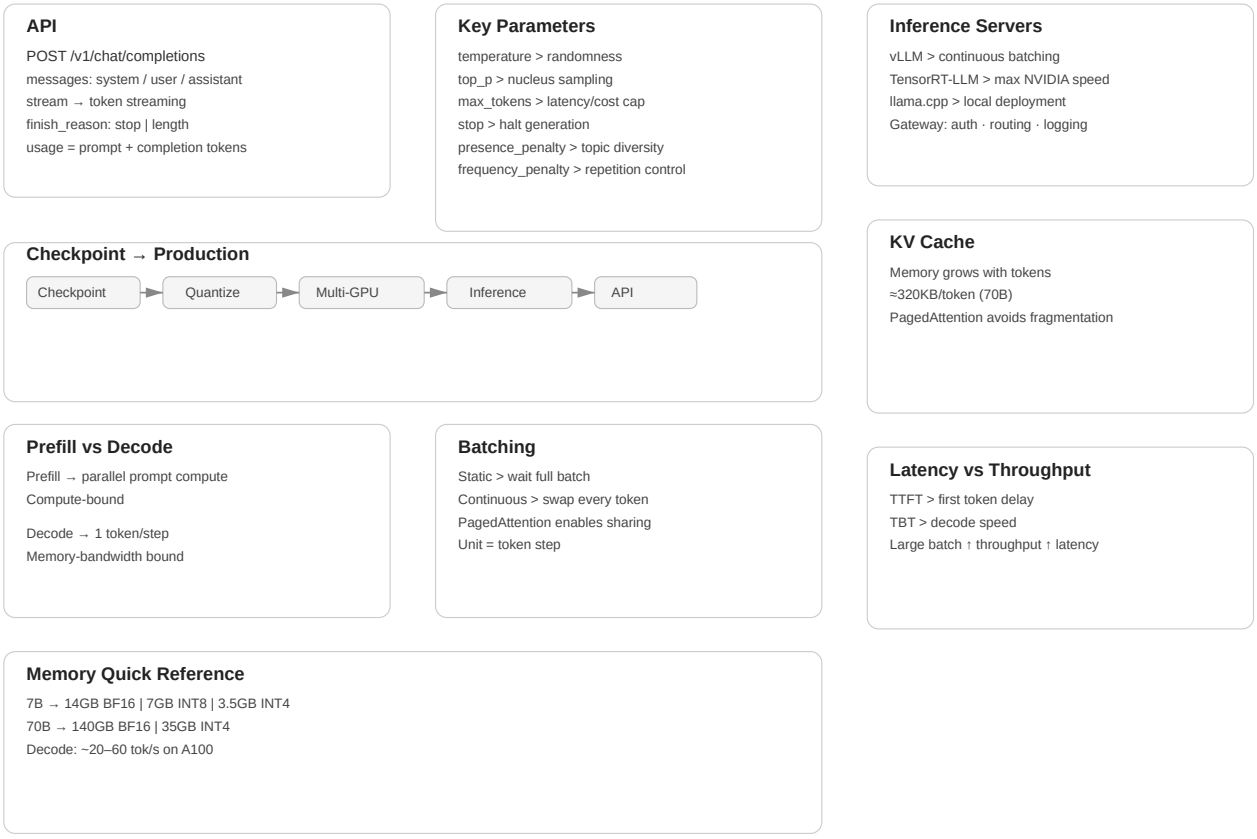


Figure 2.3: Cheat sheet.

2.9 Further reading

- Yu et al. (2022). *Orca: A Distributed Serving System for Transformer-Based Generative Models.*: Introduces continuous batching.
- Kwon et al. (2023). *Efficient Memory Management for Large Language Model Serving with PagedAttention.*: The vLLM paper; foundational for modern serving.

← Previous: 02 — Prompt engineering and structure
Next: 04 — Tokenization →

Chapter 3

Tokenization

The canonical question for this chapter: *A user sends a message while a model receive and returns numbers. How does text become numbers on the way in, and how do numbers become text on the way out?*

Figure 3.1: The journey through the Model Mind.

The request has arrived at the inference server. Before the model sees a single character, the text must be converted to numbers. This chapter covers that conversion in both directions and describes everything that can go wrong in between.

3.1 Two tokenization chapters, two different problems

If you read the training section first, you have already seen tokenization as in a batch preprocessing environment which means a corpus is tokenized once, offline, before training begins. The vocabulary is fixed. The inputs are known and errors are caught early.

Tokenization at inference is a different story. It runs on arbitrary user input, in real time, and on every request. It supposed to handle every string a user might type which could be multilingual text, source code, emoji, adversarial inputs, or a combination of all four. The encoded output feeds directly into a running model. The decoded output appears on a user's screen, sometimes token by token while they watch.

The concerns here are mostly operational, not theoretical. Understanding them is the difference between shipping reliable LLM applications or debugging corruption failures at 2am.

3.2 The encoding pipeline

When a request arrives, the first operation is encoding: converting the messages array into a flat sequence of token IDs ready for the model. This happens in two steps that are easy to confuse but are conceptually different.

3.2.1 Step 1: chat template application

The messages array has structure which includes roles, turns, and system prompts. The model has no concept of a dictionary. Before tokenization, the structured input must be flattened into a single string using a chat template.

Each model family has its own template. For LLaMA 3:

```
<|begin_of_text|>
<|start_header_id|>system<|end_header_id|>

You are a helpful assistant.<|eot_id|>
<|start_header_id|>user<|end_header_id|>

What is the capital of France?<|eot_id|>
<|start_header_id|>assistant<|end_header_id|>
```

For Mistral:

```
<s>[INST] What is the capital of France? [/INST]
```

For ChatML (OpenAI and many others):

```
<|im_start|>system
You are a helpful assistant.<|im_end|>
<|im_start|>user
What is the capital of France?<|im_end|>
<|im_start|>assistant
```

The special tokens (`<|begin_of_text|>`, `<|eot_id|>`, `<|im_start|>`) are part of the model's vocabulary and were added during fine-tuning. The model learns to associate these markers with role transitions. Feeding the wrong template to the model causes it to sees an unexpected token sequence which in turn may cause the model to ignore the system prompt, and fail to complete the turn correctly, or produces outputs that look almost right but are systematically wrong in ways that can be hard to diagnose. This is one of the most common sources of bugs: no error is thrown yet the model just quietly misbehaves.

3.2.2 Step 2: token encoding

Once the flat string is constructed, the tokenizer converts it to integer IDs. This is a deterministic step in which the same string always produces the same IDs given the same tokenizer.

```
messages = [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user",   "content": "What is the capital of France?"}
]

# Step 1: flatten with chat template
flat_string = apply_chat_template(messages, tokenizer)
# → "<|im_start|>system\nYou are a helpful assistant.<|im_end|>\n..."

# Step 2: convert to integers
token_ids = tokenizer.encode(flat_string)
# → [151644, 8948, 198, 2610, 525, 264, 10950, 17847, 13, ...]

# Step 3: pass to model
logits = model.forward(token_ids)
```

The tokenizer runs on CPU and for typical prompt lengths it is fast enough to be negligible relative to model inference time. On the other hand, for very long prompts (100k+ tokens) it can become measurable and there are some inference frameworks that cache tokenized system prompts to avoid repeating this work on every request.

3.2.3 Special tokens that require explicit handling

Several special tokens need attention beyond the template:

Beginning of sequence (BOS). Many models expect a BOS token at position 0 and the tokenizer typically adds it automatically while some serving configurations require manual insertion. A missing BOS token causes silent quality degradation which means the model's input distribution does not match training, and early tokens may be erratic.

End of turn markers. The template includes tokens marking the end of each turn and it must appear in the correct positions or the model misidentifies role boundaries.

Tool call tokens. Models fine-tuned for function calling have additional special tokens called tool call syntax. These must be in the tokenizer vocabulary and correctly placed in the template.

The generation prompt. The template typically ends with the opening of the assistant turn (e.g., `<|im_start|>assistant\n`) without closing it. This will signal the model that it should continue from here and omitting it causes the model to generate an incorrect turn structure.

3.3 Token continuation and context window management

The context window is the maximum sequence length the model can process in a single forward pass which is typically a range between 8k to 128k tokens for current models. Managing what fits in this window is a critical operational concern.

3.3.1 Count with the tokenizer, not your intuition

Token counts are not characters divided by four, or words divided by 0.75. They are the actual output of the tokenizer on the specific input string. Heuristics are inaccurate enough to cause real failures:

- Code tokenizes very differently from prose which means identifiers, operators, and whitespace have their own patterns
- Non-English text is often less efficient than English for most tokenizers
- Special tokens (role markers, tool tokens) add counts that character estimates miss entirely

Every production system that needs to stay within a context window must count tokens using the actual tokenizer, not an approximation:

```
import tiktoken

enc = tiktoken.encoding_for_model("gpt-4o")

def count_tokens(messages: list[dict]) -> int:
    num_tokens = 0
    for message in messages:
        num_tokens += 4  # role, content, separators overhead
        for value in message.values():
            num_tokens += len(enc.encode(value))
    num_tokens += 2  # reply priming tokens
    return num_tokens
```

The exact overhead is varied by model and template. Always validate against the API's reported token counts on a representative sample.

3.3.2 The context window budget

In a typical production request, the context window is shared between several competing claims:

```
Total context window (e.g., 128k tokens)
   System prompt              (fixed, e.g., 500-2,000 tokens)
   Conversation history       (grows with each turn)
   Retrieved context (RAG)    (variable, e.g., 2,000-10,000 tokens)
   Current user message       (variable)
   Reserved for completion    (max_tokens parameter)
```

The sum must not exceed the window and if it does, the request will fail or the serving layer silently truncates something which is usually the earliest conversation history, that could be the exact context the user needs the most. Applications must actively manage this budget rather than hoping it fits.

Rolling window keeps only the most recent N tokens of history. While it is simple, it loses early context which can be fine for short tasks and problematic for long conversations.

Summarization uses the model to compress earlier turns into a compact representation when history approaches the limit. With this approach it preserves semantic content at the cost of an additional model call.

Hierarchical retrieval stores full history externally and only retrieve the most relevant portions for the current turn. This approach requires a retrieval component but it can scale to arbitrarily long conversations.

3.3.3 Prompt caching and its token implications

Some APIs (Anthropic's Claude, Google's Gemini) offer explicit prompt caching: mark a portion of the prompt as cacheable, and subsequent requests that share the same prefix reuse the KV cache from the first request, charged at a reduced rate.

For cost management, structure prompts with the stable content (system prompt, long documents) first and variable content (user messages, dynamic context) in the end. Token counting for cached prompts will distinguishs between newly processed tokens and cache-read tokens, as the API reports these separately and they have completely different cost implications.

3.4 The decoding pipeline: from numbers back to text

After the model selects the next token ID (the selection process is covered in chapter "REFF"), the serving layer converts that ID back to text. This is less straightforward than it appears.

3.4.1 Detokenization

The inverse operation: token IDs to string.

```
token_ids = [15223, 374, 279, 6864, 315, 9822, 13]
text = tokenizer.decode(token_ids)
# → "Paris is the capital of France."
```

While it seems that detokenization simply involves looking up each token's string and concatenating them together, several complications exist:

Byte-level BPE and UTF-8 reconstruction. In byte-level tokenizers, tokens represent raw bytes rather than complete characters. Because UTF-8 characters are able to span multiple bytes, a single character (many Chinese characters or emojis) may be splited across several tokens. Decoding each token independently can therefore produce invalid text or replacement symbols. The correct procedure is to concatenate all token bytes first and then perform a single UTF-8 decode, ensuring the original characters are correctly reconstructed.

Special tokens in output. If the model produces a special token (end of text, role marker, tool call token), detokenization must strip it, convert it to a meaningful representation, or treat it as a halt signal. Which action is correct totally depends on context and must be handled explicitly.

3.4.2 Streaming detokenization

In streaming mode, tokens arrive one at a time and must be decoded incrementally which creates a specific problem: a byte sequence that spans multiple tokens cannot be decoded until all of its bytes have arrived.

Consider a Chinese character requiring 3 UTF-8 bytes, split across three tokens of 1 byte each:

- After token 1: 1 byte received and it is not a valid UTF-8 codepoint yet and cannot emit
- After token 2: 2 bytes received and it is still incomplete
- After token 3: 3 bytes received finally it is completed, decode and emit the character

A streaming decoder must buffer incomplete byte sequences and emit characters only once a complete Unicode codepoint is available. A naive implementations that pass `errors="ignore"` or `errors="replace"` will silently corrupt non-ASCII output while the HuggingFace `TextStreamer` and similar utilities handle this correctly.

3.5 Logprobs and their uses at inference time

Most APIs can optionally return the log probabilities of generated tokens which is more useful than it might initially appear.

3.5.1 What logprobs tell you

For each generated token, logprobs report the log probability the model assigned to the chosen token, and optionally the top-k alternatives with their probabilities:

```
{
  "token": "Paris",
  "logprob": -0.0023,
  "top_logprobs": [
    {"token": "Paris", "logprob": -0.0023},
    {"token": "Lyon",  "logprob": -6.12},
    {"token": "Nice",  "logprob": -7.88}
  ]
}
```

A logprob of -0.0023 means the model assigned probability `exp(-0.0023)` `0.998` to this token which means the model is essentially certain. A logprob of -6.12 means `exp(-6.12)` `0.002` and represent a distant second choice.

3.5.2 Classification without generation

For binary or multiple-choice tasks, logprobs let you skip generating a full response and instead read the probabilities of specific tokens directly which is faster and cheaper:

```
response = client.completions.create(
    model="gpt-4o",
    prompt=f"{text}\n\nIs this sentiment positive? Answer with yes or no.",
    max_tokens=1,
    logprobs=True,
    top_logprobs=5
)

top = response.choices[0].logprobs.top_logprobs[0]
yes_logprob = next(t.logprob for t in top if t.token.strip().lower() == "yes")
no_logprob  = next(t.logprob for t in top if t.token.strip().lower() == "no")
```

```
is_positive = yes_logprob > no_logprob
```

3.5.3 Calibration and uncertainty estimation

The perplexity of a generated response which is represented by the average negative log probability per token is a rough proxy for the model's confidence:

```
import numpy as np

logprobs = [token.logprob for token in response.choices[0].logprobs.content]
perplexity = np.exp(-np.mean(logprobs))
# High perplexity → model was uncertain about this response
```

This is not a hallucination detector which means the model can be confidently wrong, and often is. But it is a useful routing signal: responses with high perplexity can be flagged for human review or follow-up verification without running a separate evaluation model.

3.6 Common failure modes

Tokenization at inference produces specific, reproducible failure patterns. Knowing them saves significant debugging time.

Wrong tokenizer for the model. Using the GPT-2 tokenizer with a LLaMA model produces completely wrong token ID sequences. The model interprets unrelated IDs as completely different tokens which in turn leads to a garbage output.

Wrong chat template. Applying the wrong template produces subtly wrong token sequences. The model may ignore the system prompt, continue the user turn instead of starting an assistant turn, or generate role labels as content. This also fails silently and is especially confusing because outputs look almost right but are systematically off.

BOS token doubling. Some serving frameworks add BOS automatically while some expect the caller to add it. If BOS appears twice, the model sees an unexpected sequence at position 0 and may behave erratically.

Context overflow without truncation. If the total token count exceeds the context window and the serving layer does not truncate explicitly, some APIs return an error while some silently truncate from the beginning.

Encoding/decoding asymmetry. `decode(encode(s))` may not equal `s` for strings containing Unicode normalization differences, special tokens with non-standard decoding, or whitespace that the tokenizer normalizes.

3.7 Key takeaways

- Chat template application: flattening the messages array into a string with model-specific role delimiters it happens before tokenization and is model-specific; the wrong template produces silent quality degradation
- Token counting must use the actual tokenizer, not character or word heuristics; accurate counting is essential for context window management and cost control
- The context window budget must be actively managed across system prompt, conversation history, retrieved context, and completion reservation
- Streaming detokenization must buffer incomplete UTF-8 byte sequences and emit characters only once a complete codepoint is available; naive implementations corrupt non-ASCII output
- Stop sequence detection operates on decoded strings, not token IDs, and must handle sequences that span token boundaries and appear at buffer edges

- Token healing solves the boundary problem for prompts ending at arbitrary character positions, improving completion quality for structured generation
- Logprobs enable efficient classification, uncertainty estimation, and confidence- based routing without generating full responses

---

3.8 Further reading

- HuggingFace Tokenizers documentation: Fast tokenizers and chat templates.
- OpenAI Cookbook: How to count tokens with tiktoken.
- HuggingFace (2023): Chat Templating Guide.

← Previous: 03 — APIs and model serving
Next: 05 — Context windows →

Tokenization at Inference, Cheat Sheet

Encoding Pipeline

1. Apply chat template → flatten structured messages

`<system> ... <user> ... <assistant>`

2. Tokenizer converts text → token IDs

`"Paris is capital" → [1512, 374, 279]`

Critical Special Tokens

- BOS token must exist at position 0
- End-of-turn markers define roles
- Tool tokens required for function calling
- Assistant prompt signals generation start

Context Window Budget

System prompt
Conversation history
RAG context
User message + completion tokens
Always count tokens using tokenizer, never estimate.

History Management Strategies

Rolling window → keep recent tokens only
Summarization → compress earlier turns
Hierarchical retrieval → fetch relevant past context

Decoding Pipeline

Model outputs token IDs → detokenizer reconstructs UTF-8 text

`[15223, 374, 279] → "Paris is the capital"`

Byte-level tokenizers require buffering before emitting characters.

Streaming Decoding

- Tokens arrive one at a time
- Partial UTF-8 bytes must be buffered
- Emit text only when character completes

Logprobs Uses

- Confidence estimation
- Classification without generation
- Perplexity ≈ uncertainty signal

Common Failure Modes

- Wrong tokenizer for model
- Wrong chat template → silent quality loss
- Double BOS token
- Context overflow / silent truncation
- encode(decode(x)) != x due to normalization

Figure 3.2: Cheat sheet.

Chapter 4

Embeddings and Meaning Representation

The canonical question for this chapter: *How does a model convert a token ID into something that has meaning, and how does that representation evolve as it passes through the network?*

Figure 4.1: The journey through the Model Mind.

Tokenization converts the prompt text into a sequence of integer IDs. These integers has no meaning a neural network on their own. This chapter covers the step that makes them meaningful and it introduces the geometric structure of meaning that the rest of the model operates on.

4.1 The problem with integers

A token ID is an arbitrary index. The token ID for `"Paris"` might be 15342 while the token ID for `"London"` is 9562. These numbers do not carry any information about the relationship between the two cities. They are just indices in a lookup table, nothing more.

An embedding is the thing that you get after that lookup: a dense vector of floating point numbers with hundreds or thousands of dimensions. The embedding for `"Paris"` might be something like `[0.23, -0.87, 0.12, ...]` with 4096 values and the embedding for `"London"` is a completely different vector nevertheless in a well-trained model, it will be close to Paris's vector in this high-dimensional space, because the two tokens appear in similar contexts throughout the training corpus.

This is the core insight: geometric proximity in embedding space convey the semantic relatedness. Tokens that appear in similar contexts have similar embeddings. The model does not learn this rule explicitly it caused by gradient descent on the next-token prediction objective. Training discovers that representing semantically related tokens as nearby vectors produces better predictions, and adjusts accordingly over trillions of examples.

Embeddings are the bridge between the discrete world of tokens and the continuous world of neural network computation. Everything the transformer does which includes every attention operation, and every feed-forward pass operates on these continuous vectors and the model never touches the original token IDs again after the embedding lookup.

4.2 The embedding table

The embedding table is a matrix of shape `[vocab_size, d_model]`. Each row is the learned embedding for one token.

```
Embedding table (simplified, d_model = 4):

Token ID  Token     Embedding vector
0         <pad>     [ 0.00,  0.00,  0.00,  0.00]
1         <eos>     [ 0.12, -0.34,  0.87,  0.23]
2         "the"     [-0.45,  0.67, -0.12,  0.89]
3         "Paris"   [ 0.91,  0.23, -0.67,  0.45]
4         "London"  [ 0.88,  0.19, -0.71,  0.43]
5         "cat"     [-0.23,  0.89,  0.34, -0.67]
```

The lookup is a simple indexed memory access: `embedding = table[token_id]`. Implemented as a matrix multiplication with a one-hot vector in theory; in practice it is O(1) and takes microseconds.

For GPT-3 with `vocab_size = 50,257` and `d_model = 12,288`:

```
Embedding table size = 50,257 × 12,288 × 2 bytes (BF16)
                     1.24 billion parameters
                     2.47 GB
```

This is a significant fraction of total model size and it is also the most memory efficient part of the model relative to its influence: every token the model processes uses exactly one row of this table.

4.2.1 Weight tying

The same embedding table is used twice: once at the input (token IDs → embeddings) and once at the output (final hidden state → logits over vocabulary). This is called weight tying and is standard in most transformer language models.

At the output, the hidden state `h` of shape `[d_model]` is multiplied by the transposed embedding table `E^T` of shape `[d_model, vocab_size]` to produce logits of shape `[vocab_size]`:

```
logits = h @ E^T
```

Weight tying reduces the parameter count by `vocab_size × d_model` and, more importantly it enforces a geometric consistency: the direction that represents a token at the input is the same direction the model uses to predict that token at the output. This constraint improves training efficiency and generalization and it is one of those architectural decisions that looks obvious in hindsight yet it was not obvious at all beforehand.

4.3 How embeddings are learned

At the start of training, the embedding table is initialized from a small random distribution and it does not carry any semantic information they are just noise.

Through training, gradient descent updates each embedding row whenever its token appears in the training data. The update moves the embedding in the direction that reduces prediction loss. Over millions of update steps on millions of tokens, the embeddings converge to representations that capture the statistical structure of language.

The mechanism works by giving tokens that appear in similar contexts, similar gradient updates. If `"Paris"` and `"London"` both appear after `"capital of"` and before `"is beautiful"`, their embeddings are repeatedly pushed in similar directions. Over enough training, they converge to nearby vectors.

This is the distributional hypothesis, operationalized by gradient descent: *you shall know a word by the company it keeps.*

4.3.1 The rare token problem

Embedding parameters receive less frequent updates than other model parameters. A token that appears once in a billion token batch updates once while a common token might get updates in every batch. This creates an implicit learning rate inequality: frequent tokens have well trained embeddings but rare tokens may be poorly trained even in large models.

This is one reason why multilingual models struggle with low-resource languages, and why domain specific jargon may get handled poorly even in a large general model due to this simple fact that the embeddings for rare tokens have not seen enough gradient signal to settle into a useful position.

4.4 The geometry of embedding space

Trained embeddings organize into a rich geometric structure. Understanding this structure is practically useful for interpreting the model behavior and for building a retrieval system.

4.4.1 Semantic neighborhoods

Semantically related tokens cluster together. In embedding space, countries cluster near each other, verbs in the same semantic field cluster together, synonyms are close, and antonyms are often directionally opposite. This is not an artifact of any particular architecture it emerges from the distributional statistics of the language and it is similar across all model families.

4.4.2 Linear structure

The most famous property of word embeddings is the fact that analogical relationships correspond to linear operations.

```
embedding("king") - embedding("man") + embedding("woman")   embedding("queen")
embedding("Paris") - embedding("France") + embedding("Germany")   embedding("Berlin")
embedding("walked") - embedding("walk") + embedding("run")   embedding("ran")
```

Directions in embedding space encode semantic dimensions. The vector from `"man"` to `"woman"` is a gender direction. The vector from `"France"` to `"Paris"` is a capital of direction. Addition and subtraction of embedding vectors correspond to semantic composition and transformation.

This property approximately exist in static embeddings (Word2Vec, GloVe) and it continues to presist in the token embeddings of large transformers, though it might not be as clean as static embedding. The transformer layers above the embedding table further transform these representations in ways that create richer structure yet it makes it harder to extract clean linear relationships.

4.4.3 Isotropy and representation collapse

Embeddings occupy only a narrow cone of the available space, with most directions unused which makes it one of the failure modes in embedding training which is called anisotropy. When embeddings are anisotropic, cosine similarity between random embeddings is high even for semantically unrelated tokens and causes the embedding based retrieval unreliable.

Modern large models are less prone to this compared older smaller models, yet it remains a concern in fine-tuning models where the embedding distribution can shift significantly during training.

4.5 Contextual vs. static embeddings

The embedding table produces a single fixed vector for each token, regardless of context. The token `"bank"` gets the same embedding whether the surrounding context is about finance or about rivers.

This is the limitation of static embeddings an the transformer layers above the embedding table are able to solve it. After the initial embedding lookup, each transformer block modifies the token representations through attention and feed-forward operations and when it gets to the final layer, the representation of `"bank"` in a financial context is completley different from `"bank"` in a geological context. This happens not because the embedding table has changed, but because the transformer layers incorporated contextual information from surrounding tokens.

These contextual representations are called "contextual embeddings" to distinguish them from static lookup embeddings at the first layer. They are what BERT family models were designed to expose, and what RAG retrieval systems use when they need semantically rich representations.

4.5.1 The residual stream

This is a useful model for understanding how representations evolve through the network:

```
x_0 = token_embedding + positional_encoding

x_1 = x_0 + Attention_1(LayerNorm(x_0))
x_1 = x_1 + FFN_1(LayerNorm(x_1))

x_2 = x_1 + Attention_2(LayerNorm(x_1))
x_2 = x_2 + FFN_2(LayerNorm(x_2))

...

x_L = final hidden state → logits
```

Each layer adds a refinement to the running representation while the early layers tend to encode syntactic features, middle layers encode semantic relationships and the final layers encode task specific features that is used for prediction.

The residual structure means the original token embedding is always present in the stream and it gets added to every layer and never replaced. This is why the embedding table remains relevant even in very deep models: its contribution persists all the way through computation, like a base note that harmonics are built on top of.

4.5.2 Which layer to extract from

For tasks that use hidden states as embeddings, such as classification, retrieval, or similarity, a practical question that could arise is which layer should be used?

The last layer contains the most task specific information but it may over optimize for next token prediction at the expense of general semantic content. The second to last layer is a common heuristic that often outperforms the last layer for semantic similarity tasks. An average across all layers captures information at all levels of abstraction can be considered robust yet expensive. Middle layers often encode the richest structural information for syntactic tasks.

For dedicated embedding models (text-embedding-3-large, E5, BGE), this question is moot, these models are specifically trained to produce useful representations at the last layer using contrastive objectives rather than next token prediction. For retrieval tasks these models should be used instead of the generative models.

4.6 Embedding models vs. generation models

Generative models are trained to predict the next token although their internal hidden states can be extracted and used for retrieval, this is not what they are optimized for. On the other hand dedicated embedding models form a separate

class. They are trained with different objectives and are designed specifically to produce useful vector representations of text.

4.6.1 Contrastive training

Contrastive training is the dominant training approach for embedding models. The model is trained in a way that semantically similar texts have nearby embeddings and semantically different texts have distant embeddings.

The InfoNCE loss, used by models like E5, GTE, and many others:

$$\mathcal{L} = -\log\left(\frac{\exp\left(\text{sim}(q, k^+)/\tau\right)}{\sum_i \exp\left(\text{sim}(q, k_i)/\tau\right)}\right)$$

Where q is the query embedding, k^+ is the positive (semantically similar) embedding, k_i are all keys including negatives, sim is cosine similarity, and τ is a temperature parameter.

The model learns to pull positive pairs together and push negative pairs apart. Training data consists of (query, positive document) pairs which is derived from naturally occurring sources: question-answer pairs, search query logs, natural language inference datasets.

4.6.2 Hard negatives

The quality of negative examples is critical. Random negatives, which mostly means arbitrary documents from the corpus, are too easy. The model learns to distinguish clearly unrelated texts without developing fine-grained discrimination.

Hard negatives are documents that are topically related but semantically different from the positive:

```
Query:         "What is the capital of France?"
Positive:      "Paris is the capital and most populous city of France."
Easy negative: "The mitochondria is the powerhouse of the cell."
Hard negative: "France is a country in Western Europe with Paris as a major city."
               (related topic, but does not directly answer the question)
```

Mining hard negatives is itself a retrieval problem. Modern embedding model training pipelines use a previous model version or a BM25 index to find hard negatives and train a better model on those negatives, and iterate.

4.6.3 Asymmetric encoding

Many retrieval tasks are asymmetric which means that the query and the document are different in length, style, and information density. A question is short and underspecified yet the document is long and detailed.

Some embedding models handle this by encoding queries and documents with different instruction prefixes:

```
Query encoding:    embed("Represent this query for retrieval: " + query)
Document encoding: embed("Represent this document for retrieval: " + document)
```

The prefix signals to the model what role the text plays, allowing it to produce better representations for each. Models like E5 and Instructor were designed around this approach.

4.6.4 Embedding model families

Model	Dimensions	Context	Notes
text-embedding-3-small	1536	8,191 tokens	OpenAI; fast and cheap
text-embedding-3-large	3072	8,191 tokens	OpenAI; best quality in family

Model	Dimensions	Context	Notes
text-embedding-ada-002	1536	8,191 tokens	OpenAI legacy; widely deployed
E5-large-v2	1024	512 tokens	Open-source; strong performance
BGE-large-en-v1.5	1024	512 tokens	BAAI; strong on MTEB
Nomic-embed-text-v1	768	8,192 tokens	Open-source; long context
voyage-large-2	1536	16,000 tokens	Voyage AI; strong retrieval

The MTEB (Massive Text Embedding Benchmark) leaderboard is the standard reference for comparing embedding models across retrieval, classification, clustering, and semantic similarity tasks.

4.7 Similarity metrics

Embeddings are only useful if you can compare them so the choice of metric matters.

4.7.1 Cosine similarity

The standard metric for text embedding comparison:

`cosine_sim(a, b) = (a · b) / (||a|| × ||b||)`

Cosine similarity measures the angle between two vectors, ignoring magnitude. Values range from -1 (opposite directions) to 1 (identical direction). The appeal for this metric is the invariance to magnitude. A document and its summary should be similar even if the document produces a larger magnitude vector due to greater information density.

For normalized embeddings (unit vectors), cosine similarity equals the dot product: `cosine_sim(a, b) = a · b` when `||a|| = ||b|| = 1`. Most embedding APIs return normalized vectors, making the dot product the efficient choice at scale.

4.7.2 Dot product

Without normalization, the dot product combines angle and magnitude. For some retrieval tasks, magnitude carries useful signal which means a more specific or central document might have higher magnitude and it makes the raw dot product a better metric than cosine similarity.

Modern embedding models and vector databases support both; check the model documentation for which metric the model was trained to optimize.

4.7.3 Euclidean distance (L2)

This is a less common metric for text embeddings andor normalized vectors, L2 distance and cosine similarity are monotonically related:

$$\|a - b\|_2^2 = 2 - 2\cos(a, b)$$

L2 is mostly specific for applications (k-means clustering, certain index structures) where the metric properties are desirable.

4.8 Embedding dimensions and matryoshka representations

The dimensionality of an embedding determines its representational capacity and its cost. While a 3072-dimension embedding can represent finer grained semantic distinctions, it requires 12 KB per vector in float32 and more compute for similarity search. On the other hand a 768-dimension embedding has less capacity but 4× lower storage and faster search. For retrieval over millions to billions of vectors, the difference is significant.

4.8.1 Matryoshka Representation Learning (MRL)

This is a training technique that allows a single model to produce useful embeddings at multiple dimensions from the same forward pass. Instead of training separate models for different embedding sizes, MRL organizes the embedding space in a way that **useful representations exist at progressively larger prefixes of the same vector**.

MRL trains the model so that the first `d` dimensions of the full embedding are as useful as a `d`-dimensional embedding trained from scratch and simultaneously, for multiple values of `d`:

4.8.1.1 Core Idea

Let the model output a full embedding:

$$\text{embed} \in \mathbb{R}^{1536}$$

MRL trains the network such that **every prefix of the embedding** is independently meaningful:

```
- `embed[:64]` behaves like a strong 64-dim embedding
- `embed[:128]` behaves like a strong 128-dim embedding
- `embed[:256]` behaves like a strong 256-dim embedding
- ...
- `embed[:1536]` remains the highest-quality representation
```

This is analogous to **Russian matryoshka dolls**, where smaller representations are nested inside larger ones.

4.8.1.2 Training Objective

During training, the same embedding is evaluated at multiple truncation levels:

```
L = L(embed[:64]) + L(embed[:128]) + L(embed[:256]) + L(embed[:512]) + L(embed[:1536])
```

Each term computes the task loss (e.g., contrastive similarity loss) using only the first `d` dimensions.

The result is that you can use the full 1536-dimension embedding for high-accuracy search, or truncate to 256 dimensions for a 6× speedup at a small quality cost, using the same model. Dimensionality becomes a deployment parameter rather than a model choice.

OpenAI's text-embedding-3 series uses MRL, allowing callers to specify output dimensions at request time.

4.9 Embeddings in the generative inference pipeline

Within a generative LLM's inference pipeline, embeddings participate at two specific points.

4.9.1 Input: the embedding lookup

After tokenization, token IDs are converted to embeddings via the table lookup. Positional encodings are added (either as additive vectors or via RoPE modifications to attention), and result enters the first transformer block. This step is computationally trivial and takes microseconds even for long sequences.

4.9.2 Output: projection back to vocabulary

The final hidden states are projected to logit space via the transposed embedding table. This is a matrix multiplication of shape `[seq_len, d_model] @ [d_model, vocab_size]` which is the largest matrix multiply in a single forward pass for models with large vocabularies.

For a model with `d_model = 4096` and `vocab_size = 128,000`:

```
Output projection: [seq_len, 4,096] @ [4,096, 128,000]
                 = seq_len × 524,288,000 multiplications
```

For a sequence of 1,000 tokens, this is over 500 billion operations it is significant enough that some models apply the output projection only to the final token position during generation rather than to all positions.

4.10 Practical embedding operations

4.10.1 Embedding a batch of texts

```python
from openai import OpenAI

client = OpenAI()

texts = [
    "The capital of France is Paris.",
    "Paris is a city in Europe.",
    "The mitochondria is the powerhouse of the cell."
]

response = client.embeddings.create(
    input=texts,
    model="text-embedding-3-large",
    dimensions=1024  # MRL truncation
)

embeddings = [item.embedding for item in response.data]
# embeddings[0] and embeddings[1] will be much closer than embeddings[0] and [2]
```

4.10.2 Computing similarity

```python
import numpy as np

def cosine_similarity(a: list[float], b: list[float]) -> float:
    a, b = np.array(a), np.array(b)
    return float(np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b)))

sim_01 = cosine_similarity(embeddings[0], embeddings[1])
sim_02 = cosine_similarity(embeddings[0], embeddings[2])

print(f"Paris sentences:       {sim_01:.3f}")
print(f"Paris vs mitochondria: {sim_02:.3f}")
```

4.10.3 Semantic search over a corpus

```python
import numpy as np

corpus = ["doc 1 text...", "doc 2 text...", ...]
corpus_embeddings = np.array([
    client.embeddings.create(
        input=doc, model="text-embedding-3-large"
    ).data[0].embedding
    for doc in corpus
])  # shape: [n_docs, 1024]

query = "What is the capital of France?"
query_embedding = np.array(
    client.embeddings.create(
        input=query, model="text-embedding-3-large"
    ).data[0].embedding
)  # shape: [1024]

# Cosine similarity (assuming normalized embeddings from the API)
similarities = corpus_embeddings @ query_embedding  # shape: [n_docs]
top_k_indices = np.argsort(similarities)[::-1][:5]

for idx in top_k_indices:
    print(f"Score: {similarities[idx]:.3f} | {corpus[idx][:80]}")
```

For production retrieval over large corpora, replace the numpy dot product with a vector database (Pinecone, Weaviate, Qdrant, pgvector) that handles indexing and approximate nearest neighbor search at scale. The vector database is the subject of chapter “REFF”.

4.11 Key takeaways

- An embedding converts a discrete token ID into a continuous vector; geometric proximity in embedding space encodes semantic relatedness, emerging from the distributional statistics of training rather than any explicit rule
- The embedding table is a `[vocab_size, d_model]` matrix shared between input lookup and output projection (weight tying), enforcing geometric consistency between how tokens are represented and how they are predicted
- Static embeddings give every token a fixed vector regardless of context; contextual embeddings (hidden states at later layers) incorporate surrounding context through attention, the residual stream shows how representations accumulate refinements from syntactic to semantic to task-specific
- Dedicated embedding models are trained with contrastive objectives (InfoNCE loss on positive/negative pairs) rather than next-token prediction, producing representations optimized for similarity search
- Hard negatives, topically related but semantically wrong examples, are essential for training models that make fine-grained distinctions; mining them is itself a retrieval problem
- Cosine similarity is the standard metric for comparing embeddings; always use the metric the model was trained to optimize, using the wrong one degrades retrieval results in ways that are hard to diagnose
- Matryoshka Representation Learning allows a single model to produce useful embeddings at multiple dimensions, making dimensionality a deployment parameter
- The output projection (final hidden state to logits) is the most compute- intensive embedding operation in the generative forward pass; the input lookup is negligible by comparison

---

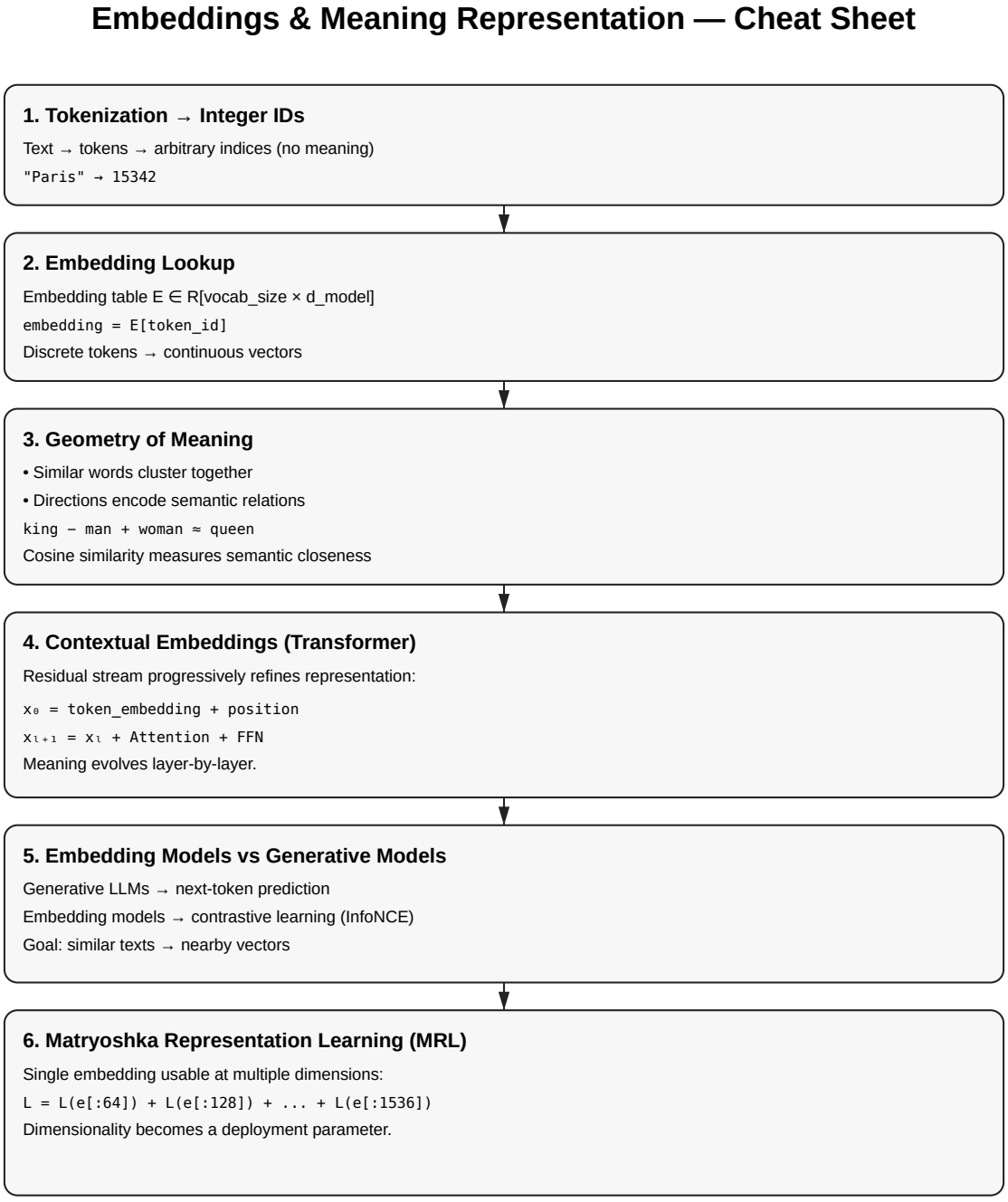


Figure 4.2: Cheat sheet.

4.12 Further reading

- Elhage et al. (2021). *A Mathematical Framework for Transformer Circuits.*: The residual stream model and how information flows through transformer layers.
- Wang et al. (2022). *Text Embeddings by Weakly-Supervised Contrastive Pre-training.*: The E5 paper; representative of modern embedding model training with hard negatives.
- Kusupati et al. (2022). *Matryoshka Representation Learning.*: MRL for flexible-dimension embeddings from a single model.
- Muennighoff et al. (2023). *MTEB: Massive Text Embedding Benchmark.*: The standard benchmark for comparing embedding models across tasks and domains.

← Previous: 05 — Context windows
Next: 07 — Transformer architecture →

Part III

The Labyrinth of Attention

Chapter 5

Transformer Architecture

The canonical question for this chapter: *Given a sequence of token embeddings, how does the transformer convert them into a prediction about what comes next and why is this architecture the one that won?*

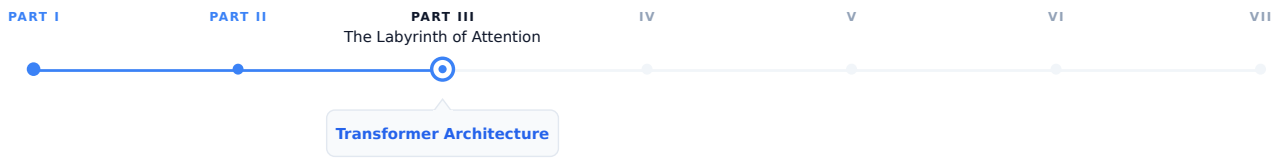


Figure 5.1: The journey through the Model Mind.

The prompt has been tokenized, the tokens have been converted to embedding vectors, and the request has arrived at the inference server. Now the model begins to think. This chapter covers the architecture that does the thinking (what it is, why it is built this way, and what each component actually does.)

5.1 Why the transformer won

Before 2017, sequence modeling was dominated by recurrent neural networks (LSTMs and GRUs). These architectures processed text one token at a time, left to right, maintaining a hidden state that carried information forward through the whole sequence. They worked, but they had two fundamental problems.

The first was the bottleneck problem, all the information about the entire input had to be compressed into a single fixed-size hidden state vector before being used to generate output. For the long sequences, early information was inevitably lost or distorted and the model had to decide what to remember and what to forget at every step.

The second was parallelization. Because each step depended on the previous step's hidden state, RNNs could not be parallelized across the sequence dimension. While the model GPU hardware are heavily tuned for parallelisim training these models were fundamentally bottlenecked by sequential computation.

The transformer, introduced in Vaswani et al. (2017), eliminated both problems with a single architectural decision: replace recurrence with attention. Every token attends directly to every other token in a single step, resulting in neither sequential dependency nor an information bottleneck, and the training can be fully parallelized across the sequence.

This is not a minor optimization but a different computational paradigm that happens to be perfectly matched to the hardware of the last decade.

5.2 The intuition before the math

Before working through the mechanics, it helps to have a mental model of what the transformer is doing at a high level.

Think of each token as a person sitting in a room with everyone else who has ever appeared in the same sequence. At each layer, every person can ask a question of every other person, receive weighted answers based on relevance, and update their understanding accordingly. After many rounds of this, each person's understanding has been shaped by the entire room.

The questions are learned queries, the relevance weighting is attention, the updating occurs in the residual stream, and the many rounds correspond to the layers.

5.3 The high-level forward pass

A transformer language model takes a sequence of token IDs as input and produces a probability distribution over the vocabulary as output in other word a prediction of what token comes next.

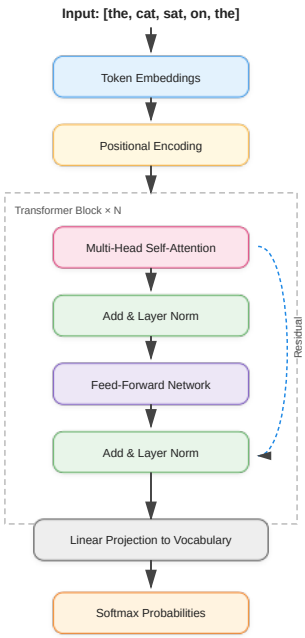


Figure 5.2: Transformer block.

Each component serves a specific function. In the following sections we work through them in order.

5.4 Token embeddings and positional encoding

5.4.1 Token embeddings

The first operation maps integer token IDs to continuous vectors using an embedding table, which is a matrix with shape `[vocab_size, d_model]` and Each row is a learned vector for that token. Looking up token ID 42 retrieves row 42 of the embedding table.

For GPT-3, `d_model = 12,288` and the vocabulary has roughly 50,000 tokens. The embedding table therefore has about 600 million parameters which is a significant fraction of the total model. These embeddings are learned end-to-end during training; the model discovers what representation of each token is most useful for predicting the next one.

The embedding table is shared with the output layer. The same matrix used to look up input embeddings is used (transposed) to project the final hidden state back to vocabulary space. This weight tying reduces parameters and improves performance by enforcing geometric consistency between how tokens are represented and how they are predicted.

5.4.2 Positional encoding

Attention, as we will see, is permutation-invariant which means if you shuffle the tokens in the input, attention produces the same output for each token regardless of order. This is a problem: `"the cat sat"` and `"sat cat the"` would be processed identically. Positional encodings inject order information into the representations by adding a position-dependent signal to each token embedding.

5.4.3 Sinusoidal encoding (the original transformer)

It uses a deterministic functions:

```
PE(pos, 2i)   = sin(pos / 10000^(2i / d_model))
PE(pos, 2i+1) = cos(pos / 10000^(2i / d_model))
```

Different dimensions oscillate at different frequencies that produces a unique fingerprint for each position. These encodings are fixed and generalizes to sequence lengths not seen during training.

5.4.4 Learned positional embeddings**

they replace the formula with a second learned matrix of shape `[max_sequence_length, d_model]`, each position gets its own trained vector, updated by gradient descent.

5.4.5 Rotary Positional Embeddings (RoPE)

They are widely used in modern transformer architectures such as LLaMA, PaLM, and many recent open-weight large language models. Unlike absolute positional encodings, which are added directly to token embeddings before the attention mechanism, RoPE injects positional information directly into the self-attention computation itself.

The core idea behind RoPE is to encode position by applying a rotation to the query and key vectors in each attention head. For each token position, the representation is split into pairs of dimensions, and each pair is rotated by an angle that depends on the token's absolute position. This rotation is structured so that when computing the dot product between a query at position `m` and a key at position `n`, the result depends on the relative offset `(m - n)` rather than their absolute positions.

More formally, RoPE treats each pair of dimensions in a query or key vector as a 2D plane and applies a rotation matrix whose angle increases linearly with position. For a given position `p`, each 2D subspace is rotated by:

$$\theta_p = p \cdot \omega_i$$

where ω_i is a frequency term that depends on the embedding dimension index. This design closely relates to sinusoidal encodings, but differs fundamentally in that the positional information is applied as a transformation of the query and key vectors rather than an additive signal.

In practice, each pair of dimensions is rotated using sine and cosine functions:

$$\begin{pmatrix} x_1' \\ x_2' \end{pmatrix} = \begin{pmatrix} \cos(\theta_p) & -\sin(\theta_p) \\ \sin(\theta_p) & \cos(\theta_p) \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$$

This rotation preserves vector magnitude while changing orientation, embedding positional structure geometrically into the representation space.

A key consequence of this formulation is that the dot product between a query at position `m` and a key at position `n` naturally depends on the relative distance `(m - n)`. This emerges from trigonometric identities that cause the absolute positional components to cancel out during the inner product. As a result, RoPE encodes relative position information implicitly without explicitly computing pairwise distances.

This property provides several important benefits:

- Firstly, it captures relative position information directly, which is often more meaningful for language modeling than absolute position. Language understanding typically depends on relationships between tokens rather than their absolute locations in a sequence.
- Secondly, RoPE improves extrapolation to longer sequences. Because the rotation is a smooth, continuous function of position, it generalizes beyond the training context window more gracefully than learned absolute embeddings. However, performance can still degrade for very long contexts if not trained appropriately.
- Thirdly, RoPE introduces no additional learned parameters for positional encoding. The positional structure is fully deterministic, making it parameter-efficient and reducing the risk of overfitting positional patterns.

In multi-head attention, each head applies the same rotational mechanism but operates in different learned subspaces. This allows different heads to specialize in different positional behaviors. Some heads naturally focus on local dependencies, while others capture long-range relationships due to accumulated phase differences.

5.4.6 ALiBi (Attention with Linear Biases)

It takes a very different approach to positional information. Instead of explicitly encoding positions into token representations, ALiBi modifies the attention mechanism itself by adding a fixed bias to attention scores based on the distance between tokens.

In ALiBi, the attention score between a query at position `m` and a key at position `n` is penalized linearly according to their distance `|m - n|`. This means that tokens that are farther apart receive a progressively stronger negative bias, encouraging the model to focus more on nearby context while still allowing access to long-range dependencies when needed.

A key advantage of ALiBi is that it requires no positional embeddings at all—neither learned nor fixed sinusoidal ones. Instead, positional information is entirely encoded through the attention bias structure. This simplicity makes ALiBi highly efficient and robust, particularly for long-context extrapolation. It has been used in models such as MPT and has shown strong performance in settings where generalization to longer sequences is important.

For a query at position m and a key at position n, the attention score is adjusted as:

$$\text{score}(m, n) = q_m \cdot k_n - \alpha_h \cdot |m - n|$$

Here: - $q_m \cdot k_n$ is the standard dot-product attention score - $| m - n |$ is the absolute distance between tokens - α_h is a head-specific slope (each attention head gets a different decay rate)

One of the key design choices in ALiBi is that different attention heads use different slopes. Some heads have a steep penalty (they focus very locally), while others have a shallow penalty (they can attend further away). This creates a

natural diversity of attention ranges without explicitly designing multiple mechanisms. The slopes are not learned, Instead, they are fixed using a deterministic scheme that assigns geometrically spaced values. With this method it avoids additional parameters and ensures predictable extrapolation behavior.

5.5 Self-attention: the core mechanism

Self-attention is an operation that allows every token to gather information from every other token in the sequence. It is the defining innovation of the transformer.

5.5.1 The intuition

Consider: `"The cat slept on the bed because it was dark."`

What does `"it"` refer to? For a human, it's clear that `"it"` refers to the environment, not `"the cat"` or `"the bed"`. A model needs to make this connection to process the sentence correctly. Self-attention allows the token `"it"` to look at every other token, assign an attention weight to each one (higher weight to the context suggesting darkness, lower weight to unrelated tokens like `"bed"`), and blend their representations into its own. After attention, the representation of `"it"` carries information about the surrounding environment being dark.

5.5.2 Queries, keys, and values

Self-attention uses three learned linear projections of each token's embedding: Query (Q), Key (K), and Value (V).

For a sequence of `n` tokens with embedding dimension `d_model`:

$$
\begin{aligned}
Q &= XW_Q \qquad shape : [n, d_k] \\
K &= XW_K \qquad shape : [n, d_k] \\
V &= XW_V \qquad shape : [n, d_v]
\end{aligned}
$$

Where `X` is the matrix of token embeddings and W_Q, W_K, W_V are learned weight matrices.

The attention scores are computed as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Breaking this down:

1. QK^T is dot product between every query and every key. The shape `[n, n]` and an entry `[i, j]` measures how much token `i` should attend to token `j`.
2. $\sqrt{d_k}$ scales the dot products by the square root of the key dimension. Without this, dot products in high dimensions become very large and pushes the softmax into regions with near-zero gradients and destabilizing training.
3. $softmax()$ converts scores to probabilities. Each row sums to 1. This is the attention weight matrix.
4. $\times V$ is the weighted sum of value vectors. Each token's output is a blend of all value vectors, weighted by how much it attends to each position.

The result is a new representation for each token that has been informed by the entire sequence. The attention chapter REFF goes deeper into the computational mechanics and what attention heads actually compute.

5.5.3 The causal mask

For language model training, the model must not attend to future tokens. When predicting the token at position `i`, only positions `0` through `i-1` should be visible, otherwise the model is simply reading the answer.

This is enforced by a causal mask: before softmax, a large negative value (effectively `-∞`) is added to attention scores where `j > i`. After softmax, these positions have attention weight 0.

```
Tokens:
Position:   0     1     2     3
          the   cat   sat    on

Causal Attention Matrix (upper triangle masked)

                Key tokens →
              the   cat   sat    on
Query ↓
the           1.0   0.0   0.0   0.0
cat           0.3   0.7   0.0   0.0
sat           0.1   0.5   0.4   0.0
on            0.2   0.1   0.3   0.4
```

Each token attends to itself and all previous tokens and it will never see the future tokens. This masking enables a decoder-only Transformer to function as an autoregressive language model. It allows the model to be trained on the full sequence in a single forward pass, while ensuring that each position can attend only to preceding tokens.

5.5.3.1 Bidirectional attention (encoder models)

Used in encoder-only models such as BERT, RoBERTa, and most embedding models.

Here, **no causal mask is applied**:

- each token can attend to all other tokens
- both left and right context are visible

This enables richer representations because every token is contextualized by the full sequence.

However, it breaks autoregressive generation:

> if a model can see future tokens, it cannot learn to predict them

Instead, encoder models are trained with masking objectives (e.g., masked language modeling), where some tokens are hidden and must be reconstructed.

5.5.3.2 Prefix masking (prefix-LM / partial bidirectionality)

A hybrid scheme used in some long-context and instruction models.

The sequence is split into two parts:

- **prefix (context)**: fully visible to itself and often to the entire sequence
- **generation region (suffix)**: causal masking applies

This allows:

- bidirectional understanding of the prompt/context
- autoregressive generation for outputs

It is particularly useful for: - long document conditioning - retrieval-augmented generation - structured prompt formats

---

5.5.3.3 Encoder–decoder (cross-attention masking)

The encoder–decoder transformer is designed for tasks where an input sequence must be *understood* before a new sequence is generated. Models such as **T5** and **BART** follow this structure.

The architecture separates the model into two cooperating systems:

- The **encoder** reads the entire input at once using bidirectional self-attention. Every token can attend to every other token, allowing the model to construct a contextual representation which is often described as a *memory* of the input.
- The **decoder** then generates the output sequence step by step. Unlike the encoder, its self-attention is **causally masked**, meaning each token can only attend to previously generated tokens. This preserves autoregressive generation.

  A second attention mechanism (**cross-attention**) connects the two halves of the model. Here, the decoder forms queries while the encoder provides keys and values, allowing generation to remain grounded in the encoded input representation.

This separation of *understanding* and *generation* makes the encoder–decoder architecture especially effective for transformation tasks such as translation, summarization, and structured rewriting. This creates a structured information flow:

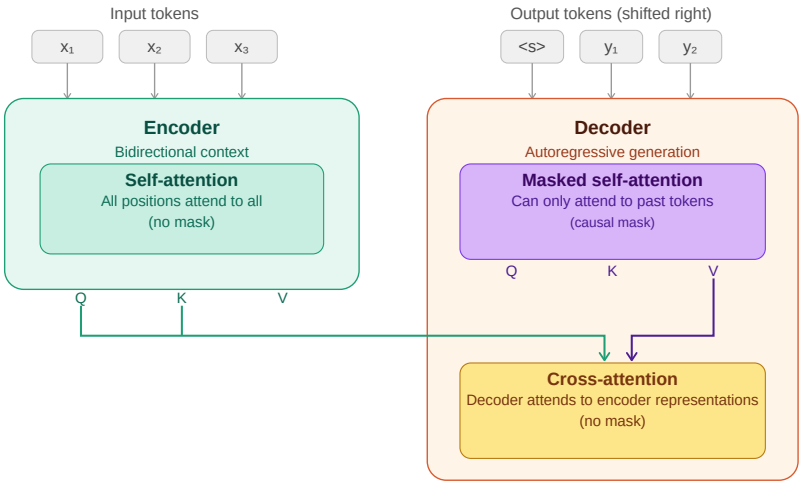


Figure 5.3: encoder-decoder information flow

5.6 Multi-head attention

A single attention operation gives each token one way of relating to every other token nevertheless tokens simultaneously relate to each other in multiple ways.

In `"John gave Mary the book"`, `John` relates to `gave` syntactically (as subject), to `Mary` semantically (as giver to recipient), and to `the book` as the transferred object. A single attention head cannot capture all of these simultaneously and it results in production of a single weighted blend over the sequence.

Multi-head attention runs multiple attention operations in parallel, each with its own learned Q, K, V projections. With this format each head can specialize in a different type of relationship. Finally, the outputs of all heads are concatenated and projected through a learned output matrix W_O back to d_{model} dimensions.

For GPT-3: `d_model = 12,288`, `h = 96` heads, `d_k = 128` per head. The 96 heads run in parallel, each attending to the sequence from a different perspective.

5.7 The feed-forward network

After the attention sublayer, each token passes through a position-wise feed-forward network (FFN) independently:

$$FFN(x) = GeLU(xW_1 + b_1)W_2 + b_2$$

The intermediate dimension is typically 4× the model dimension: for `d_model = 1,024`, the FFN expands to 4,096 dimensions, applies the activation, then projects back to 1,024.

Crucially, the FFN operates on each token independently and identically. The attention layer mixes information across tokens; the FFN processes each token in isolation. Together they form a complementary pair: attention gathers the context and FFN processes it.

5.8 Residual connections and layer normalization

5.8.1 Residual connections

```
x = x + Attention(LayerNorm(x))
x = x + FFN(LayerNorm(x))
```

The output of each sublayer is added back to its input which gives the gradients a direct path backward through the network, enabling training of very deep models. Without residual connections, training a 96-layer network is extremely difficult due to gradients vanishing or exploding before reaching the earlier layers.

Residual connections make the default behavior of each sublayer simply pass its input through unchanged. As a result, sublayers only need to learn small refinements on top of the identity function, which is a much easier optimization problem than learning the full transformation from scratch.

5.8.2 Layer normalization

Layer normalization is used to stabilize the activations of a transformer by normalizing each token's representation across its feature dimensions. Unlike normalization methods that have been used in convolutional networks, it operates independently on each token rather than across the batch or sequence.

Formally, for a hidden state vector $x \in \mathbb{R}^d$:

$$\text{LayerNorm}(x) = \frac{x - \text{mean}(x)}{\sqrt{\text{var}(x) + \varepsilon}} \cdot \gamma + \beta$$

where: - `mean(x)` and `var(x)` are computed over the feature dimension - ε is a small constant for numerical stability - γ and β are learned scale and shift parameters and allow the model to restore or reshape the normalized representation if needed.

This normalization keeps activations in a controlled range, which prevents instability as signals propagate through many stacked layers. Without it, deep transformers become difficult to train due to exploding or vanishing activations in the residual stream.

5.8.2.1 Why not Batch Normalization?

Batch Normalization is rarely used in transformer language models for a fundamental reason: it depends on batch-level statistics.

In BatchNorm, normalization is computed across examples in a minibatch. This creates several problems for language modeling:

- **Inference mismatch**: at inference time, batch sizes are often 1 (especially in autoregressive decoding), making batch statistics unreliable or unavailable
- **Sequence variability**: token sequences have variable lengths, making batch-wise statistics inconsistent across steps
- **Autoregressive constraint**: generation happens one token at a time, so there is no meaningful "batch" context during decoding

Because of these issues, BatchNorm introduces instability and is incompatible with the sequential nature of language model inference. LayerNorm avoids this entirely by normalizing within each token independently.

5.8.2.2 Pre-LN vs Post-LN

The original transformer used **Post-LN**, where normalization is applied after the residual connection:

```
x = x + Sublayer(x)
x = LayerNorm(x)
```

Modern large language models instead use **Pre-LN**, where normalization happens before the sublayer:

```
x = x + Sublayer(LayerNorm(x))
```

Pre-LN is now the standard architecture because it significantly improves optimization stability in deep networks. While it can sometimes slightly reduce raw performance compared to Post-LN in certain setups, it provides much more reliable training at scale. It ensures that each sublayer receives well-conditioned inputs while maintaining a clean gradient flow through the residual stream. This stability benefit becomes increasingly important as models grow to hundreds of layers, where training dynamics would become difficult to control.

5.8.2.3 RMSNorm (modern simplification)

Some recent architectures replace LayerNorm with **RMSNorm (Root Mean Square Normalization)**, used in models such as LLaMA.

RMSNorm simplifies LayerNorm by removing the mean-centering step:

$$\text{RMSNorm}(x) = \frac{x}{\sqrt{\text{mean}(x^2) + \varepsilon}} \times \gamma$$

Key differences: - no subtraction of the mean - normalization is based only on vector magnitude - slightly cheaper computationally - empirically comparable or better performance in large-scale models

Despite its simplicity, RMSNorm preserves the key property needed for deep transformers: controlling the scale of activations in the residual stream.

5.9 Stacking transformer blocks

A single transformer block gives the model one round of attention and one round of FFN computation while modern LLMs stack many of them:

Model	Layers	d_model	Heads	Parameters
GPT-2 Small	12	768	12	117M
GPT-2 XL	48	1,600	25	1.5B
GPT-3	96	12,288	96	175B
LLaMA 2 70B	80	8,192	64	70B
LLaMA 3 405B	126	16,384	128	405B

Each layer refines the representation produced by the previous layer. Early layers tend to encode syntactic features while middle layers encode semantic relationships and final layers encode task-specific features relevant to prediction. This hierarchy

5.10 Architecture variants

The standard transformer has been modified in various ways by recent models.

5.10.1 Grouped Query Attention (GQA)

Standard multi-head attention uses separate Q, K, and V projections for each head. In contrast, Multi-Query Attention (MQA) shares a single set of K and V projections across all heads, while keeping separate Q projections per head. Grouped Query Attention (GQA) works in a middle ground: K and V are shared across groups of heads, while each head (or each group of heads) retains its own Q projections.

LLaMA 2 70B, LLaMA 3, and Mistral use GQA. The motivation is inference efficiency: KV cache memory (chapter REFF) scales with the number of K, V heads and fewer KV heads is equal to smaller cache, resulting in longer sequences and larger batches at the same memory budget.

5.10.2 Sliding window attention

Used by Mistral 7B. Instead of attending to all previous tokens, each token attends only to a local window of the most recent `w` tokens. For long sequences this reduces attention complexity from $O(n^2)$ to $O(n \cdot w)$. Information from beyond the window propagates through the layered structure of the network and multiple layers of local attention can capture long-range dependencies indirectly.

5.10.3 Activation functions (ReLU → Tanh → Sigmoid → GELU → Swish → GLU → SwiGLU)

The feed-forward network (FFN) in transformers is where most of the non-linear feature transformation happens and over time, activation functions have evolved from simple thresholding operations to smooth functions and finally to gated multiplicative mechanisms.

5.10.3.1 ReLU (Rectified Linear Unit)

$$\text{ReLU}(x) = \max(0, x)$$

ReLU is the simplest activation function, zeroing out all negative values while leaving positive values unchanged, which makes it computationally efficient and encourages sparse activations. However, this abrupt cutoff at zero can discard useful information, making it a useful starting point but not ideal for training deep transformer models.

5.10.3.2 Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

The sigmoid function was one of the earliest nonlinearities used in neural networks, mapping inputs into the range ((0, 1)) in a smooth and differentiable way that can be interpreted as a probability or gating mechanism; intuitively, it acts like a soft switch that controls how much of a signal passes through, from fully blocked at 0 to fully allowed at 1.

5.10.3.3 Tanh (Hyperbolic Tangent)

$$\tanh(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$$

Tanh was widely used in early neural networks and recurrent models before transformers, producing outputs in the range ([-1, 1]) with a smooth, zero-centered, symmetric shape that preserves both positive and negative signals; unlike ReLU, it does not discard negative information but instead compresses inputs into a bounded range, allowing both excitatory and inhibitory effects to be represented.

5.10.3.4 Why sigmoid and tanh are rarely used in transformers

Both sigmoid and tanh share a key limitation in that they saturate for large absolute values of (x), causing gradients to vanish in those regions and making optimization difficult in deep architectures such as transformers. Despite this, sigmoid remains important in practice, especially in gating mechanisms like GLU and LSTM-style components, as well as in attention-related soft selection functions where smooth probabilistic control is useful.

5.10.3.5 GELU (Gaussian Error Linear Unit)

$$\text{GELU}(x) = x \cdot \Phi(x)$$

GELU replaces hard thresholding with a smooth probabilistic gating function.

Intuition: Instead of “drop or keep”, it asks: > “how likely is this feature to be useful?”

This smoothness improves optimization stability and is used in BERT and early GPT-style models.

5.10.3.6 Swish

Swish is a simpler, learned-smooth alternative to GELU:

$$\text{Swish}(x) = x \cdot \sigma(x)$$

where ((x)) is the sigmoid function.

Swish is a smooth, fully differentiable activation similar in spirit to GELU, but simpler in form and notable for being non-monotonic, meaning it can both slightly suppress or enhance features depending on the input value. Intuitively,

rather than using hard gating like ReLU or probabilistic gating like GELU, Swish applies a self-gated mechanism where each feature determines how much of itself to pass forward, which is why it often matches or slightly improves GELU's performance while remaining conceptually simpler.

5.10.3.7 GLU (Gated Linear Unit)

instead of simply modifying the activation function, GLU introduces explicit gating between two learned projections, where the input is split into a content stream and a gate stream:

$$\mathrm{GLU}(x) = (xW_a) \odot \sigma(xW_b)$$

This design separates "what to pass" from "how much to pass," enabling multiplicative feature control that is substantially more expressive than standard pointwise activations.

5.10.3.8 SwiGLU (Swish-Gated Linear Unit)

Modern LLMs (LLaMA, PaLM, Mistral) use SwiGLU, which replaces the sigmoid gate with Swish:

$$\mathrm{FFN}(x) = (\mathrm{Swish}(xW_1) \odot (xW_2))W_3$$

SwiGLU combines the smooth, self-gated nonlinearity of Swish with the multiplicative two-stream structure of GLU, so rather than simply applying a nonlinearity to a feature, it learns how two projected feature streams interact through multiplicative gating. This shift from pointwise transformation to learned feature interaction makes it significantly more expressive than standard activation functions.

5.10.3.9 Visual intuition: from tanh to GELU-like behavior

A useful way to build intuition for modern activation functions is to see how they evolve from simple bounded nonlinearities into smooth, ReLU-like gating mechanisms that combine stability with adaptive feature control.

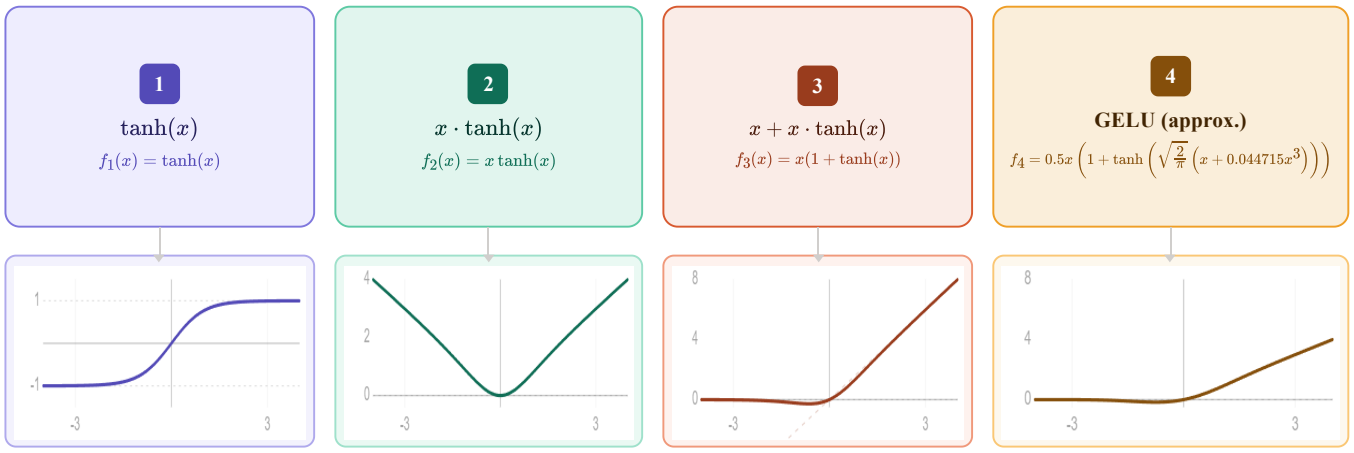


Figure 5.4: Tanh to GELU.

5.10.3.10 Visual intuition: from sigmoid to Swish-like behavior

A useful way to build intuition for modern activation functions is to see how they evolve from simple probabilistic gating functions into smooth, self-modulated nonlinearities. Sigmoid starts as a soft on–off switch, squeezing inputs

into ((0,1)), while Swish generalizes this idea by letting the input itself be modulated by a smooth gate, producing a function that behaves like a continuous, input-dependent scaling mechanism rather than a fixed probabilistic filter.

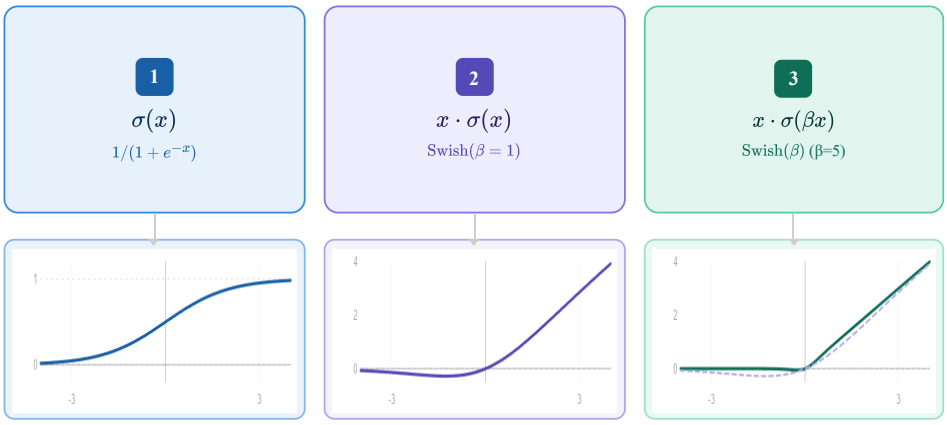


Figure 5.5: Sigmoid to swish.

5.10.4 Mixture of Experts (MoE)

Mixture of Experts (MoE) is used by Mixtral, reportedly GPT-4, it is a sparse Transformer architecture that increases model capacity without proportional increase in computation. It replaces the standard Feed-Forward Network (FFN) with multiple parallel FFNs (experts) and uses a learned routing mechanism to activate only a small subset per token.

Formulation

Given a set of experts $\text{FFN}_1, \ldots, \text{FFN}_N$, a gating function computes routing scores:

$$g(x) = \text{softmax}(W_r x)$$

MoE enables sparse computation, where only a fraction of parameters are active per token. This allows model capacity to scale significantly while keeping per-token compute similar to dense models. For example, a model with 8 experts of 7B parameters (~47B total) may activate only 1–2 experts per token, resulting in compute comparable to a much smaller dense model.

5.11 The output layer

After the final transformer block, the hidden state at the last token position is projected back to vocabulary size:

$$\text{logits} = h_{\text{final}} W_{\text{embed}}^{\top} \quad \text{shape: [vocab\_size]}$$

This produces one unnormalized score per vocabulary token. Softmax converts logits to probabilities:

$$P(\text{token}_i) = \frac{\exp(\text{logits}_i)}{\sum_j \exp(\text{logits}_j)}$$

The training objective (cross-entropy loss) maximizes the probability assigned to the actual next tokenand that is the only thing the model is explicitly trained to do. Chapter REFF covers training objectives in detail.

5.12 What the model actually learns

The transformer is trained to predict the next token. That is all. There is no explicit objective to learn grammar, facts, reasoning, or common sense, yet models trained at scale acquire all of these.

Why? Because predicting the next token well requires understanding everything that determines what comes next. Next-token prediction is a seems to be a simple objective that implicitly rewards a comprehensive world model. In order to predict `"Paris"` after `"The capital of France is"`, the model must have encoded the fact that Paris is the capital of France or to predict grammatically correct continuations, it must have internalized syntactic rules.

5.13 Key takeaways

- The transformer replaced recurrence with attention, enabling full parallelism during training and direct token-to-token interaction at any distance
- Token embeddings map integer IDs to continuous vectors; positional encodings inject order information; RoPE encodes relative rather than absolute position
- Self-attention allows every token to gather information from every other token via query-key dot products, scaled and softmaxed into attention weights over value vectors
- The causal mask enforces that tokens cannot attend to future positions, enabling autoregressive training on the full sequence in a single forward pass
- Multi-head attention runs multiple attention operations in parallel, each specializing in a different type of relationship.
- The feed-forward network processes each token independently after attention, acting as associative memory for facts learned during training
- Residual connections and layer normalization stabilize training of very deep networks; Pre-LN and RMSNorm are the current standard
- Modern variants (RoPE, GQA, SwiGLU, MoE) improve efficiency and performance without changing the fundamental architecture
- The next-token prediction objective implicitly rewards a comprehensive world model because everything that makes text predictable (grammar, facts, reasoning) improves prediction quality

---

5.14 Further reading

- Vaswani et al. (2017). *Attention Is All You Need.*: The original transformer paper.
- Alammar, J. (2018). *The Illustrated Transformer.*: The clearest visual explanation of the architecture available. Essential companion to this chapter.

← Previous: 06 — Embeddings and meaning representation
Next: 08 — Attention computation →

Transformer Architecture — Cheat Sheet

Why Transformers Won

- RNNs: sequential → no parallelism
- Long-range info bottleneck in hidden state
- Transformers: attention → full parallelism + direct token interaction

High-Level Pipeline

Tokens → Embeddings → Positional Encoding → Transformer Blocks → Logits → Softmax

`P(next token | context)`

Embeddings & Position

Token embedding: lookup table (vocab → vectors)

Weight tying: input embedding = output projection

Positional encodings:

- Sinusoidal (fixed, original transformer)
- Learned embeddings
- RoPE: rotates Q/K → relative position

Self-Attention

`Q = XWq, K = XWk, V = XWv`

`Attention = softmax(QKᵀ / √d) V`

Each token attends to all previous tokens

Creates contextualized representation

Attention Masks

- Causal (decoder): only past tokens visible
- Bidirectional (encoder): full sequence visible (BERT)
- Encoder-decoder: cross-attention between memory + generation
- Prefix LM: prefix bidirectional, suffix causal

Multi-Head Attention

Multiple parallel attention heads

Each head learns different relation type

Outputs concatenated + projected

`Specialization emerges from training`

Feed-Forward Network

`FFN(x) = W2 · activation(W1 x)`

Per-token independent transformation

Modern activations:

- GELU (default GPT/BERT)
- SwiGLU (modern LLaMA, Mistral)

Residual + Normalization

x = x + Sublayer(LayerNorm(x)) (Pre-LN)

Stabilizes deep training

LayerNorm → RMSNorm (modern models)

Pre-LN = stable gradients at scale

Modern Variants

- RoPE: relative position encoding
- GQA: shared K/V across heads
- MoE: sparse expert FFNs
- Sliding window attention (Mistral)

Output Layer

`logits = h Wᵀ → softmax → P(next token)`

Training objective: next-token prediction only

Figure 5.6: Cheat sheet.

Chapter 6

Attention Computation

The canonical question for this chapter: *When the model processes your prompt, how does each token decide what to look at and what does the hardware actually do to compute that?*

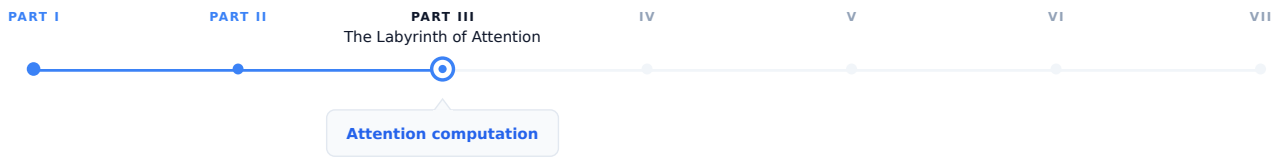


Figure 6.1: The journey through the Model Mind.

In the previous chapter we introduced attention conceptually: queries, keys, values, dot products, softmax, weighted sum. This chapter covers what attention computation actually looks like on real hardware, under the constraints of latency, memory bandwidth, and concurrent requests.

6.1 The quadratic problem

Standard self-attention has a fundamental scaling property where both computation and memory grow quadratically with sequence length. For a sequence of `n` tokens:

Attention score matrix:	$n \times n$
Memory for scores:	$n^2 \times 2$ bytes (BF16)
Compute for scores:	$n^2 \times d_k$ multiplications ($QK^\top$)
Compute for output:	$n^2 \times d_v$ multiplications (scores $\times V$)

At small sequence lengths this is manageable:

Sequence length	Score matrix	Memory
512 tokens	512 × 512	0.5 MB
2,048 tokens	2,048 × 2,048	8 MB
8,192 tokens	8,192 × 8,192	128 MB
32,768 tokens	32,768 × 32,768	2 GB

Sequence length	Score matrix	Memory
131,072 tokens	131,072 × 131,072	32 GB

At 128k tokens on the other hand which is currently supported by several models the attention score matrix alone would require 32 GB of GPU memory per layer per head which means a model with 96-layer and 96 heads would need $96 \times 96 \times 32$ GB = 295 TB and that is clearly impossible.

In reality the score matrix is never actually materializes in practice with the help of FlashAttention but the compute still scales quadratically. Doubling the sequence length quadruples the attention computation and it is central cost driver for long-context inference.

6.2 What the GPU actually does during attention

Understanding attention at the hardware level requires understanding two main bottlenecks in GPU computation, arithmetic throughput and memory bandwidth. **Arithmetic throughput** refers to how many floating-point operations per second the GPU can perform (an H100 delivers approximately 1,000 TFLOPS for BF16 matrix multiplications). **Memory bandwidth** refers to how fast data can be read/write from/to GPU high bandwidth memory (an H100 provides approximately 3.35 TB/s). The ratio between these, known as arithmetic intensity, determines whether a computation is compute-bound or memory-bandwidth-bound.

For matrix multiplication, the main operation in attention and FFNs, large matrices have high arithmetic intensity and are compute-bound, while small matrices or element-wise operations have low arithmetic intensity and are memory-bandwidth-bound.

During the decoding phase (processing one new token against the KV cache), attention is severely memory-bandwidth-bound. In this phase GPU must read the entire KV cache (potentially gigabytes of data) to compute attention scores for a single new token. The arithmetic work is tiny relative to the memory access while the GPU's massive arithmetic throughput sits idle, waiting for data. This is the reason why decoding throughput scales with memory bandwidth, not arithmetic throughput and why increasing sequence length primarily increases memory pressure and not the compute pressure, during generation.

6.3 Standard attention: the naive computation

Before FlashAttention, attention was computed in the obvious way:

```
# Standard attention (naive)
# Q: [seq_len, d_k]
# K: [seq_len, d_k]
# V: [seq_len, d_v]

scores = Q @ K.T                          # [seq_len, seq_len] - written to HBM
scores = scores / math.sqrt(d_k)          # scaling - read/write from/to HBM
scores = scores + causal_mask             # masking - read/write from/to HBM
scores = softmax(scores, dim=-1)          # softmax - read/write from/to HBM
output = scores @ V                       # [seq_len, d_v] - read/write from/to HBM
```

Each step read/write from/to HBM. The score matrix of shape `[seq_len, seq_len]` is materialized in HBM multiple times during scaling, masking, and softmax.

For a sequence of 8,192 tokens in BF16:

- Score matrix: 8,192 × 8,192 × 2 bytes = 128 MB

- Each read/write of the score matrix: 128 MB / 3.35 TB/s 38 microseconds
- Across multiple reads and writes: hundreds of microseconds in memory traffic alone

The actual arithmetic is fast, while the movement of data between compute units and HBM is the bottleneck, which making this a classic I/O-bound computation.

6.4 FlashAttention

FlashAttention (Dao et al., 2022) computes exact attention which is basically same numerical result as standard attention, while using dramatically less memory and running significantly faster. The trick is reorganizing the computation to minimize HBM traffic.

6.4.1 The key insight: tiling

FlashAttention divides the Q, K, and V matrices into tiles that fit in SRAM, the fast on-chip memory sitting directly next to the compute units, with much higher bandwidth than HBM but much smaller capacity:

```
SRAM bandwidth:  ~20 TB/s   (fast, ~50 MB on H100)
HBM bandwidth:   ~3.35 TB/s (slower, 80 GB on H100)
```

By loading tiles into SRAM and performing the full attention computation on those tiles before writing results back to HBM, FlashAttention reduces HBM reads and writes from $O(n^2)$ to $O(n)$ while the score matrix is never written to HBM at all.

6.4.2 Online softmax

The challenge: softmax requires knowing the maximum value across all scores in a row before normalizing while in the tiled computation, you process one tile at a time without seeing the full row. How do you compute correct softmax without materializing the full score matrix?

FlashAttention uses an online softmax algorithm that maintains running statistics (the running maximum and running sum of exponentials) and updates them as each tile is processed. After all tiles, the statistics are complete and the output is correctly normalized:

```
running_max = -infinity
running_sum = 0
running_output = zeros(d_v)

for each key/value tile (K_tile, V_tile):
    scores_tile = Q_row @ K_tile.T / sqrt(d_k)
    tile_max = max(scores_tile)

    # Update running statistics
    new_max = max(running_max, tile_max)
    running_sum = (running_sum * exp(running_max - new_max)
                   + sum(exp(scores_tile - new_max)))
    running_output = (running_output * exp(running_max - new_max)
                      + exp(scores_tile - new_max) @ V_tile)
    running_max = new_max

# Final normalization
output = running_output / running_sum
```

This procedure produces the identical result to naive attention while never storing more than one tile's worth of scores at a time.

6.4.3 FlashAttention in practice

FlashAttention is 2–4× faster than standard attention on typical sequence lengths, with the speedup increasing as sequences grow longer and more importantly, its memory usage is O(n) rather than O(n^2).

6.5 Multi-head attention at inference

Multi-head attention runs `h` independent attention operations in parallel. For a model with `h = 96` heads and `d_model = 12,288`:

$$d_k = d_v = \frac{d_{\text{model}}}{h} = 128 \quad \text{(per head)}$$

Projections:

$$Q : [n,\, 12288] \,@\, [12288,\, 96 \times 128] \;\rightarrow\; [n,\, 96,\, 128]$$
$$K : [n,\, 12288] \,@\, [12288,\, 96 \times 128] \;\rightarrow\; [n,\, 96,\, 128]$$
$$V : [n,\, 12288] \,@\, [12288,\, 96 \times 128] \;\rightarrow\; [n,\, 96,\, 128]$$

Per-head attention:

$$\text{scores} = \frac{Q_h K_h^\top}{\sqrt{128}} \;\rightarrow\; [n,\, n]$$

$$\text{output} = \text{softmax}(\text{scores})\, V_h \;\rightarrow\; [n,\, 128]$$

Concatenate: $[n,\, 96,\, 128] \;\rightarrow\; [n,\, 12288]$

Output proj: $[n,\, 12288] \,@\, [12288,\, 12288] \;\rightarrow\; [n,\, 12288]$

In practice, all 96 heads are computed in parallel by treating the head dimension as a batch dimension. The Q, K, V projections produce batched tensors processed simultaneously by the matrix multiplication kernel.

6.5.1 MHA vs. MQA vs. GQA

Multi-Head Attention (MHA) uses separate Q, K, and V projection matrices for each head. While this increases expressiveness, it is memory-intensive because each head requires its own stored K and V tensors in the KV cache.

Multi-Query Attention (MQA) (Shazeer, 2019): All attention heads share a single set of K and V projections, while Q remains separate per head. This significantly reduces KV cache usage by a factor of h, for example, a 96-head model achieves a ~96× smaller cache and in general, this comes with only a modest impact on output quality.

MHA: $Q \in \mathbb{R}^{n \times h \times d_k},\ K \in \mathbb{R}^{n \times h \times d_k},\ V \in \mathbb{R}^{n \times h \times d_v} \;\Rightarrow\; \text{KV cache} \propto 2 \cdot h \cdot d_k$

MQA: $Q \in \mathbb{R}^{n \times h \times d_k},\ K \in \mathbb{R}^{n \times 1 \times d_k},\ V \in \mathbb{R}^{n \times 1 \times d_v} \;\Rightarrow\; \text{KV cache} \propto 2 \cdot d_k$

Grouped-Query Attention (GQA) (Ainslie et al., 2023): the middle ground in which Heads are divided into groups; each group shares one K and V projection:

Component	Shape	Details
Q (Query)	$[n, 96, d_k]$	per head
K (Key)	$[n, 8, d_k]$	per group

Component	Shape	Details
V (Value)	$[n, 8, d_v]$	per group
KV Cache	$\propto 2 \times 8 \times d_k$	12× smaller than MHA

LLaMA 2 70B uses GQA with 8 KV heads and Mistral 7B uses GQA with 8 KV heads. Quality loss compared to MHA is negligible on standard benchmarks while the KV cache reduction is significant for serving.

6.6 The KV cache at inference

During decoding, the model generates one token at a time and at each step, this new token must attend to all previous tokens. Recomputing the K and V vectors for all previous tokens at every step would be extremely wasteful due to the fact that those tokens have not changed, so their projections have not changed either.

The KV cache stores K and V tensors for all previous tokens and reuses them at each generation step. Only the new token's Q, K, V vectors need to be computed fresh. This is covered in full detail in chapter REFF nevertheless the momory cost understanding will be discussed here

6.6.1 KV cache memory per token

For LLaMA 2 70B (L=80 layers, 8 KV heads, d_k=128, BF16):

Per token per layer: $2 \times 8 \times 128 \times 2$ bytes = 4 KB

Per token total: 4×80 layers = 320 KB

For a 4,096-token sequence: 320 KB × 4,096 **1.28 GB of KV cache** per sequence. At batch size 64 concurrent sequences: 64 × 1.28 GB = 82 GB which is more than the 80 GB on an H100. This is the reason KV cache management is the central serving constraint for long-context workloads.

6.6.2 Prefill vs. decode attention

During prefill (processing the input prompt), all tokens' K and V vectors are computed simultaneously and written to the KV cache it is a batch matrix multiplication, efficient and compute-bound. On the othe hand during decoding (generating output), each new token's K and V are computed and appended to the cache, then attention is computed between that single new token's Q and the full KV cacheand it is a vector-matrix multiply, memory-bandwidth-bound:

```
Prefill: [n_prompt, d_k] @ [d_k, n_prompt]      → large matmul, compute-bound
Decode:  [1, d_k] @ [d_k, n_prompt + n_gen]     → vector-matrix, bandwidth-bound
```

This asymmetry is why decoding is slower per token than prefill, and why disaggregated serving (separate hardware for prefill and decode phases) is considered as an active area of optimization.

6.7 Sparse and linear attention

The quadratic cost of standard attention has motivated significant research into more efficient alternatives yet None has been able to displace full attention in frontier models, but several are worth understanding.

6.7.1 Sparse attention

Instead of every token attending to every other token, sparse attention restricts each token to a subset of positions:

Local window attention: each token attends only to the surrounding `w` tokens which reduces the complexity to O(n × w). With this approach the information still can propagate across long distances through multiple layers of local attention.

Strided attention: alternating layers use different patterns in which some layers use local windows and some use strided patterns (attend every k steps) allowing attention to distant tokens. Used in Longformer and BigBird.

Sliding window with global tokens: certain tokens (typically the first ones) attend to all other tokens and are attended to by all others. These global tokens propagate long-range information while the rest of the sequence uses local attention.

The limitation: the sparsity pattern must be specified in advance. If it does not match the actual information dependencies in the data, quality degrades. Full attention is flexible that helps the model to learn any pattern which is why it has remained dominant.

6.7.2 Linear attention

The key idea behind **linear attention** is to reinterpret softmax attention as a kernel operation whose structure allows the order of computation to change.

Ignoring scaling and normalization constants for clarity, the softmax attention weight between tokens can be written as

$$A_{ij} = \exp(q_i^\top k_j).$$

The attention output for token i therefore becomes

$$\text{output}_i = \frac{\sum_j \exp(q_i^\top k_j)\, v_j}{\sum_j \exp(q_i^\top k_j)}.$$

This reveals that attention is fundamentally a **kernel similarity** $\kappa(q_i, k_j) = \exp(q_i^\top k_j)$ meaning attention computes a weighted kernel regression over values.

With linear attention we can assume that this exponential kernel can be approximated by an inner product in a feature space, $\exp(q^\top k) \approx \phi(q)^\top \phi(k)$, where the feature map $\phi : \mathbb{R}^d \to \mathbb{R}^{d_\phi}$ embeds queries and keys into a space where their similarity become linear and substituting this approximation into attention gives

$$\text{output}_i = \frac{\sum_j \phi(q_i)^\top \phi(k_j)\, v_j}{\sum_j \phi(q_i)^\top \phi(k_j)}.$$

Because $\phi(q_i)$ does not depend on j, it can be factored outside the sums:

$$\text{output}_i = \frac{\phi(q_i)^\top \sum_j \phi(k_j) v_j^\top}{\phi(q_i)^\top \sum_j \phi(k_j)}.$$

All interactions with the sequence are now contained in two global statistics,

$$S = \sum_j \phi(k_j) v_j^\top, \qquad z = \sum_j \phi(k_j),$$

so the output becomes

$$\text{output}_i = \frac{\phi(q_i)^\top S}{\phi(q_i)^\top z}.$$

Stacking all queries yields the linear attention form

$$\boxed{\text{Attn}(Q,K,V) = \phi(Q)(\phi(K)^\top V)} \quad \text{(up to normalization)}.$$

The crucial consequence is that computation can be reordered:

$$(QK^\top)V \;\longrightarrow\; \phi(Q)(\phi(K)^\top V),$$

allowing the key-value product to be computed once and reused for all queries.

Method	Formula	Complexity
Standard attention	$\text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$	$O(n^2)$
Linear attention	$\phi(Q)(\phi(K)^\top V)$	$O(n)$

The limitation is that the approximation works for some tasks but degrades on others, particularly tasks requiring precise comparison of specific tokens. Softmax produces a peaky distribution (concentrating attention on a small number of tokens) which is important for retrieval-like tasks that linear approximations cannot replicate faithfully.

6.7.3 State space models and hybrid architectures

Another direction for scaling sequence models is to move away from attention completely and instead model sequences through **learned dynamical systems**. State space models (SSMs) replace pairwise token interaction with a recurrent state that evolves over time, allowing sequences to be processed in linear time.

Rather than computing interactions between all tokens, an SSM maintains a compact hidden state updated sequentially,

$$h_t = Ah_{t-1} + Bx_t, \qquad y_t = Ch_t,$$

so information is carried forward through the state instead of recomputed via an attention matrix. Models such as Mamba achieve $O(n)$ computation and memory while retaining long-range dependency modeling, since past context is compressed into the evolving state.

Hybrid architectures combine both paradigms. Instead of choosing between attention and recurrence, models like Jamba and Griffin interleave attention layers with SSM layers. Attention provides flexible global reasoning when precise token interactions matter, while SSM blocks handle long-context processing efficiently by propagating information through state updates.

These hybrid designs represent an active research direction: as context lengths grow, future architectures may rely less on dense attention and more on mixtures of attention, recurrence, and continuous-time sequence modeling.

6.8 Ring attention and sequence parallelism

For extremely long sequences, even FlashAttention with $O(n)$ memory may exceed a single GPU's capacity with the help of Ring attention the sequence distributions can go across multiple GPUs.

Each GPU holds a slice of the Q, K, and V matrices. The attention computation proceeds in a ring: each GPU computes a partial attention using its local Q slice against the current K/V slice, then passes the K/V to the next GPU while receiving the previous GPU's K/V:

```
Ring of 4 GPUs:

Step 1: GPU0 computes Q0×K0V0, GPU1 computes Q1×K1V1, ...
        K/V rotate: K0V0 → GPU1, K1V1 → GPU2, ...

Step 2: GPU0 computes Q0×K3V3 (received from GPU3)
        GPU1 computes Q1×K0V0 (received from GPU0), ...

Step 4: Each GPU has accumulated contributions from all K/V slices
        → complete attention output for its Q slice
```

After one full revolution, each GPU has computed its complete attention output using K/V from all other GPUs and enables sequence lengths that scale linearly with the number of GPUs.

6.9 Attention in the decode loop: step by step

Here is exactly what happens during one attention computation step during decoding, after prefill is complete and the model is generating token by token:

```
State at step t:
  KV cache contains K, V for tokens 0 through t-1
  New token t has been embedded with positional encoding applied

1. Compute Q, K, V for token t:
   Q_t = x_t @ W_Q    → [1, d_k] per head
   K_t = x_t @ W_K    → [1, d_k] per head  (stored in KV cache)
   V_t = x_t @ W_V    → [1, d_v] per head  (stored in KV cache)

2. Append K_t, V_t to KV cache:
   K_cache[:t+1] = [K_0, K_1, ..., K_{t-1}, K_t]

3. Compute attention scores:
   scores = Q_t @ K_cache[:t+1].T / sqrt(d_k)    → [1, t+1] per head

4. Softmax:
   attn_weights = softmax(scores)    → [1, t+1] per head

5. Weighted sum:
   attn_output = attn_weights @ V_cache[:t+1]    → [1, d_v] per head

6. Concatenate heads and project:
   output = concat(attn_output_per_head) @ W_O    → [1, d_model]

7. Add to residual stream:
   x_t = x_t + output
```

Step 3 is the memory-bandwidth bottleneck: it reads the entire KV cache from HBM. At 4,096 tokens for LLaMA 2 70B, step 3 reads approximately 1.28 GB of KV cache per generation step. At 3.35 TB/s bandwidth, that is 0.38 milliseconds per step (for attention alone, before FFN and other operations). This is why long-context generation is slower per token than short-context generation.

6.10 Key takeaways

- Attention computation scales quadratically with sequence length in both memory and compute; this is the central cost driver for long-context inference, at 128k tokens, the naive score matrix would require 32 GB per layer per head
- Standard attention materializes the full $n \times n$ score matrix in HBM, causing excessive memory traffic; FlashAttention eliminates this by tiling the computation into SRAM and using online softmax, reducing HBM access from $O(n^2)$ to $O(n)$
- During decode, attention is memory-bandwidth-bound: the GPU reads the entire KV cache for a single new token; throughput scales with HBM bandwidth, not arithmetic throughput
- GQA shares K and V projections across groups of heads, reducing KV cache by the ratio of total heads to KV heads, the primary reason GQA is now the default for new models
- Prefill is compute-bound (large batch matrix multiply); decode is memory- bandwidth-bound (vector-matrix multiply against the growing KV cache), the same model, two different hardware bottlenecks
- Sparse attention and linear attention reduce $O(n^2)$ cost but sacrifice flexibility; state space models and hybrid architectures are competitive alternatives for extreme sequence lengths

---

6.11 Further reading

- Dao et al. (2022). *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness.*: The foundational paper.
- Shazeer (2019). *Fast Transformer Decoding: One Write-Head is All You Need.*: The original Multi-Query Attention paper.
- Ainslie et al. (2023). *GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints.*: Grouped-Query Attention.
- Liu et al. (2023). *Ring Attention with Blockwise Transformers for Near-Infinite Context.*: Ring attention for distributed long-context computation.
- Gu & Dao (2023). *Mamba: Linear-Time Sequence Modeling with Selective State Spaces.*: The leading SSM alternative to attention.

← Previous: 07 — Transformer architecture
Next: 09 — KV cache →

Core Attention Equation

$\text{Attention}(Q,K,V) = \text{softmax}(QK^T / \sqrt{d_k})\, V$

QK^T → similarity scores between tokens

softmax → attention weights (probability distribution)

V → content being mixed based on weights

Tensor Shapes

Q: $[n, d_k]$

K: $[n, d_k]$

V: $[n, d_v]$

QK^T: $[n, n]$ (attention matrix)

Output: $[n, d_v]$

Intuition

Each token asks: "what should I look at?"

Q = query (what I want)

K = key (what I offer)

V = value (content to pass)

Output = weighted memory retrieval

Causal Mask (Decoder Attention)

Prevents access to future tokens

Implemented by adding $-\infty$ to invalid positions before softmax

$j > i \rightarrow \text{mask}(i,j) = -\infty \rightarrow \text{softmax} = 0$

Ensures autoregressive generation: predict next token only from past

GPU Reality

Two bottlenecks:

- Compute (TFLOPS) → fast matrix math
- Memory bandwidth → KV cache dominates decode

Decode is memory-bound: GPU waits for KV cache, not compute

KV Cache

Stores past Keys & Values

Avoids recomputation every step

Memory grows with sequence length:

$O(n \times \text{layers} \times \text{heads} \times d_k)$

Bottleneck for long-context inference

FlashAttention

Key idea: never store full n×n matrix

Tiling into SRAM (fast on-chip memory)

Online softmax computes statistics incrementally

Result: same output, much less memory traffic

$O(n^2$ memory → $O(n)$ HBM traffic)

Multi-Head Attention

Parallel attention heads

Each head learns different relation type

Concat heads → linear projection

`Specialization emerges from training`

Attention Variants

- MHA: full expressivity, high KV cost
- MQA: shared K/V, minimal cache
- GQA: grouped sharing (LLaMA, Mistral)
- Sparse/Linear: efficiency vs quality tradeoff

Figure 6.2: Cheat sheet.

Chapter 7

KV Cache

The canonical question for this chapter: *Generating one token requires attending to every previous token. How do we avoid recomputing all of that work on every single generation step and what does managing that memory look like in a production system?*

Where are we?

Figure 7.1: The journey through the Model Mind.

The previous chapter showed how attention is computed in inference and this chapter focuses on the engineering implications of that process when generation happens sequentially: at each step, the model must retain access to the keys and values from all previously generated tokens. While the KV cache makes this computationally practical, it becomes the primary memory management challenge in production scale LLM serving.

7.1 The problem that KV cache solves

Autoregressive generation is fundamentally sequential and generating each token requires attending to all previously generated tokens. To generate token `t`, the model must attend to all preceding tokens `0` through `t-1`, and generating each subsequent token expands the context by one, requiring the model to repeatedly process the entire accumulated sequence.

Without caching, generating a 500 token response to a 100 token prompt would require:

```
Step 1:   process 101 tokens → generate token 101
Step 2:   process 102 tokens → generate token 102
...
Step 500: process 600 tokens → generate token 600
```

Total compute proportional to $101 + 102 + ... + 600 \approx 175,000$ token processing steps. For large models, this process is highly inefficient: by step 500, most of the computation power is spent to reprocess tokens that were already handled

at step 1, recomputing the same key and value vectors even though the underlying inputs have not changed.

The KV cache eliminates this redundancy by computing key and value vectors only once and stores them in memory. At each new generation step, the model computes the new token's Q, K, and V vectors, retrieves the cached keys and values for all previous tokens, and performs attention over the full cached context.

With the KV cache:

```
Prefill:          process all 100 prompt tokens once, store their K and V
Each decode step: compute K, V for one new token, append to cache, attend
Total work:       proportional to 100 (prefill) + 500 (one step per token)
```

The savings grow with sequence length. Without the KV cache, practical LLM deployment at today's context lengths would be economically infeasible.

7.2 What is stored

For each token in the sequence, for each transformer layer, for each KV head, the cache stores two vectors: the key and value produced by the linear projections of that token's hidden state.

```
KV cache structure:

Layer 0:
  K: [seq_len, n_kv_heads, d_k]   ← key vectors for all tokens so far
  V: [seq_len, n_kv_heads, d_v]   ← value vectors for all tokens so far

Layer 1:
  K: [seq_len, n_kv_heads, d_k]
  V: [seq_len, n_kv_heads, d_v]

...

Layer L-1:
  K: [seq_len, n_kv_heads, d_k]
  V: [seq_len, n_kv_heads, d_v]
```

Query vectors are not cached because they are used only once: the query for token `t` is needed solely to compute attention at generation step `t` and becomes irrelevant afterward. On the other hand, the key and value vectors for token `t` are reused at every subsequent step until generation ends. The KV cache exploit this asymmetry by storing persistent keys and values while recomputing only the transient query vectors.

7.3 Attention with and without a KV cache

Consider a single attention head. After processing a sequence of length (t),

$$K_t = \begin{bmatrix} k_1 \\ k_2 \\ \vdots \\ k_t \end{bmatrix}, \quad V_t = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_t \end{bmatrix}.$$

To generate the next token, the model first computes the hidden state (h_{t+1}) and projects it to

$$q_{t+1} = W_Q h_{t+1}, \quad k_{t+1} = W_K h_{t+1}, \quad v_{t+1} = W_V h_{t+1}.$$

The attention output is

$$o_{t+1} = \text{softmax}\left(\frac{1}{\sqrt{d_k}} q_{t+1} \begin{bmatrix} K_t & k_{t+1} \end{bmatrix}^T\right) \begin{bmatrix} V_t & v_{t+1} \end{bmatrix}.$$

The important observation is that all previous keys and values already exist:

$$K_t = [k_1, \dots, k_t], \quad V_t = [v_1, \dots, v_t].$$

Only the final column k_{t+1}, v_{t+1} must be computed and appended.

7.3.1 Without caching

At step $t+1$, the model recomputes

$$\{k_1, \dots, k_t, k_{t+1}\}, \quad \{v_1, \dots, v_t, v_{t+1}\}.$$

The cost of the key/value projections is therefore $O(t+1)$. Across an entire generation of length T,

$$\sum_{t=1}^{T} O(t) = O(T^2).$$

7.3.2 With caching

The projections

$$k_i = W_K h_i, \quad v_i = W_V h_i$$

are computed only once when token i is created. During generation of step $t+1$, the model only needs to compute the new projections k_{t+1}, v_{t+1} which results in $O(1)$ projection cost per step. Over a full sequence of length T, this accumulates to $\sum_{t=1}^{T} O(1) = O(T)$.

The attention computation itself still grows linearly with context length, but the repeated projection cost has been eliminated.

7.4 Memory arithmetic

KV cache memory is deterministic and computable from model hyperparameters and sequence length:

`KV cache bytes = 2 × L × n_kv_heads × d_k × seq_len × bytes_per_element`

Where `2` is for K and V, `L` is the number of transformer layers, `n_kv_heads` is the number of KV heads (equals `n_heads` for MHA, fewer for GQA), `d_k` is the key/value dimension per head, and `bytes_per_element` is 2 for BF16.

7.4.1 Examples

LLaMA 2 7B (L=32, n_kv_heads=32, d_k=128, BF16):

```
Per token: 2 × 32 × 32 × 128 × 2 = 524,288 bytes = 512 KB
4,096 tokens: 512 KB × 4,096 = 2 GB
```

LLaMA 2 70B (L=80, n_kv_heads=8, d_k=128, BF16):

```
Per token: 2 × 80 × 8 × 128 × 2 = 327,680 bytes = 320 KB
4,096 tokens:  320 KB × 4,096  = 1.28 GB
32,768 tokens: 320 KB × 32,768 = 10 GB
```

GPT-3 175B (L=96, n_kv_heads=96, d_k=128, BF16):

```
Per token: 2 × 96 × 96 × 128 × 2 = 4,718,592 bytes   4.5 MB
4,096 tokens: 4.5 MB × 4,096 = 18 GB
```

The GPT-3 numbers explain why GQA was adopted: reducing `n_kv_heads` from 96 to 8 reduces the per-token cache from 4.5 MB to 375 KB (12 times reduction) with modest quality loss. GQA is now standard in nearly every new model architecture for exactly this reason.

7.4.2 The memory budget

A serving instance has a fixed GPU memory budget, divided between competing claims:

```
Total GPU memory (e.g., 80 GB H100)
   Model weights     (fixed, e.g., ~35 GB for LLaMA 2 70B in BF16 with GQA)
   Activation memory (fixed per batch, ~1-3 GB)
   KV cache          (variable that grow with sequence length and batch size)
   Reserved          (CUDA overhead, fragmentation buffer, ~3 GB)

Available for KV cache: 80 - 35 - 2 - 3   40 GB
```

For LLaMA 2 70B with 40 GB available:

```
Max tokens in cache: 40 GB / 320 KB   125,000 tokens

Batch size 32, avg sequence 4,096 tokens:
  32 × 4,096 = 131,072 tokens → barely fits

Batch size 32, avg sequence 8,192 tokens:
  32 × 8,192 = 262,144 tokens → does not fit
```

This is the core serving constraint: KV cache capacity limits concurrency at long sequence lengths. Every additional token in context competes directly with additional concurrent users for the same GPU memory.

7.5 The lifecycle of a KV cache entry

Understanding the sequence of operations that touch the KV cache during a single request makes the memory management problem concrete.

7.5.1 Prefill

When a request arrives, the full prompt is tokenized and processed in a single forward pass (the prefill phase). For each layer, K and V are computed for every prompt token simultaneously and written to the KV cache:

```
Input:  [tok_0, tok_1, tok_2, ..., tok_n]  ← full prompt
Output: KV cache populated for positions 0 through n
        Model output: logits at position n → first generated token
```

Prefill is compute-bound and processing all prompt tokens in parallel is a large batch matrix multiply that saturates GPU arithmetic throughput. It is the phase where prompt length most directly affects latency: time to first token scales with prompt length.

7.5.2 Decode

Each decode step generates exactly one new token:

```
1. Embed new token → hidden state
2. Compute K, V for new token → append to KV cache
3. Compute Q for new token
4. Attend: Q against full KV cache (all positions 0 through current)
5. FFN and residual → logits → sample next token
```

Step 4 is memory-bandwidth-bound. The GPU reads the entire KV cache to compute attention for a single query vector. Arithmetic work is trivial compared to the memory transfer. This is why decode throughput is limited by HBM bandwidth, not by compute, and why longer sequences are slower per token.

7.5.3 Request completion

When generation ends (stop token, max_tokens, stop sequence), the KV cache entries for that request are freed. In a naive contiguous allocator this means marking a region of memory as available. In PagedAttention it means returning individual pages to the free pool.

7.6 Naive allocation and its problems

The simplest KV cache allocation strategy: at request admission, allocate a contiguous block of memory sized for the maximum possible output length. Hold it until generation completes, then free it.

This produces two forms of waste:

Internal fragmentation. If the request was allocated 8,192 token positions but only generated 200 tokens, the remaining 7,992 positions were reserved but unused for the lifetime of the request. At 320 KB per token for LLaMA 2 70B, that is 2.55 GB of unusable memory per such request.

External fragmentation. As requests arrive and complete with variable lengths, the free memory becomes fragmented into non-contiguous gaps. A new request requiring 1 GB may fail to be admitted even if 2 GB is technically free, because no single contiguous 1 GB region is available.

On real workloads with variable request lengths, naive allocation wastes 60–80% of available KV cache memory.

7.7 PagedAttention

PagedAttention (Kwon et al., 2023) solves the fragmentation problem by borrowing the virtual memory paging idea from operating systems.

Instead of allocating a contiguous block per request, PagedAttention divides the KV cache into fixed-size pages which typically has 16 tokens per page. A page table maps logical token positions to physical page locations. Pages are allocated on demand as generation proceeds and freed immediately on completion.

```
Page table for three concurrent requests:

Request A (generating, 48 tokens so far):
  Logical positions 0-15  → Physical page 3
  Logical positions 16-31 → Physical page 7
  Logical positions 32-47 → Physical page 12

Request B (generating, 32 tokens so far):
  Logical positions 0-15  → Physical page 1
  Logical positions 16-31 → Physical page 9

Request C (generating, 64 tokens so far):
  Logical positions 0-15  → Physical page 2
  Logical positions 16-31 → Physical page 4
  Logical positions 32-47 → Physical page 5
  Logical positions 48-63 → Physical page 11

Free pages: [0, 6, 8, 10, 13, 14, 15, ...]
```

Physical pages are non-contiguous. Logical positions map to whatever physical page is available. A new request needs four pages for a 64-token sequence; it gets four pages from the free pool regardless of where they are in physical memory.

Internal fragmentation is bounded to at most one partially filled page per request (the current page being written). External fragmentation is eliminated entirely and all free pages are usable regardless of their physical location. PagedAttention achieves over 90% KV cache utilization on typical workloads, compared to 20–40% for naive contiguous allocation.

7.7.1 Prefix sharing

PagedAttention enables a further optimization: memory sharing between requests with a common prefix.

If two requests share the same system prompt like a 1,000-token system prompt that appears in every request to a production API, their KV cache pages for that prefix are identical. There is no reason to store them twice.

With prefix sharing, the pages for the common prefix are marked as shared and reference-counted. Multiple requests point to the same physical pages via their page tables:

```
Request A page table:
  Positions 0-15   → Physical page 7  (shared system prompt, page 1)
  Positions 16-31  → Physical page 8  (shared system prompt, page 2)
  Positions 32-47  → Physical page 12 (private request A's user message)

Request B page table:
  Positions 0-15   → Physical page 7  (shared same physical pages)
  Positions 16-31  → Physical page 8
  Positions 48-63  → Physical page 15 (private request B's user message)
```

When both requests complete, the shared pages are not freed until all references are released. This is copy-on-write semantics: pages are shared until a write is required, at which point a private copy is made.

For a 1,000-token system prompt with 16 tokens per page: 63 pages of KV cache are shared across all concurrent requests using that prompt. At 320 KB per token for LLaMA 2 70B, this is 320 MB of KV cache that does not need to be recomputed or stored per-request.

7.7.2 Radix attention

Radix attention (SGLang, Zheng et al., 2023) generalizes prefix sharing to arbitrary prefix trees rather than just common prefixes.

In applications with structured multi-step prompts system like agentic workflows, few-shot templates, and multi-document retrieval, requests may share prefixes at multiple points in their structure, not just at the beginning. Radix attention maintains a trie (prefix tree) of all cached KV sequences. When a new request arrives, the longest matching prefix in the trie is found and reused:

```
Prefix tree:
  [System prompt] → shared by all requests
       [Document A] → shared by requests querying document A
           [Query 1] → private to request 1
           [Query 2] → private to request 2
       [Document B] → shared by requests querying document B
            [Query 3] → private to request 3
```

This turns prefix sharing from a single-level optimization into a general caching system for the KV cache. For agentic applications where many requests share substantial common context, radix attention can dramatically reduce both memory and compute.

7.8 KV cache quantization

The KV cache can be stored in reduced precision to trade a small amount of quality for significant memory savings.

INT8 KV cache stores key and value vectors as 8-bit integers rather than 16-bit floats. This provides a 2× memory reduction. Quality loss is typically negligible for standard generation tasks K and V vectors have bounded ranges as outputs of linear projections of layer-normalized hidden states, making them well-suited to quantization.

FP8 KV cache uses 8-bit floating point rather than integer quantization. H100 GPUs have native FP8 support. Better precision characteristics than INT8 for values with non-uniform distributions, with the same 2× memory reduction.

INT4 KV cache reduces to 4 bits for a 4× memory reduction. More quality loss; requires careful calibration. Used in memory-constrained scenarios where the alternative is rejecting requests.

The quality tradeoff for KV cache quantization is typically smaller than for weight quantization for two reasons: quantization errors in the KV cache are averaged across many attention heads (reducing their impact), and modern calibration techniques can minimize the effect on attention score distributions.

INT8 KV cache quantization is now standard in production and is supported by vLLM, TensorRT-LLM, and most major inference frameworks. The 2× memory savings effectively doubles serving concurrency on fixed hardware.

7.9 KV cache eviction

Some applications require sequences longer than the KV cache can hold. Rather than rejecting these requests, The serving system can evict cached entries to make room for new ones, at the cost of a quality trade-off.

7.9.1 Eviction policies

Sliding window evicts the oldest tokens, keeping only the most recent `w` token positions in cache. Simple to implement. Degrades quality for any task requiring long-range context, the model simply cannot attend to evicted positions.

H2O (Heavy-Hitter Oracle) evicts tokens with the lowest cumulative attention scores. Tokens that have received little attention weight in recent steps are unlikely to be needed in future steps. Empirically preserves quality better than sliding window eviction at the same cache budget.

Attention sink preservation always keeps the BOS (beginning of sequence) token regardless of other eviction decisions. The BOS token functions as an attention sink and many heads route excess attention weight to it when no

other token is highly relevant. Evicting the BOS token causes attention distributions to become erratic and output quality to degrade unpredictably. Any eviction policy should treat the BOS token as eviction-exempt.

SnapKV identifies important KV vectors per head using attention patterns from a recent observation window, then evicts the remaining positions. Can maintain quality at 10–20% of the full KV cache size for long documents make it useful for very long context workloads where even INT8 quantization is insufficient.

7.10 Key takeaways

- The KV cache stores key and value vectors for all previous tokens, avoiding quadratic recomputation cost; query vectors are not cached because they are used exactly once per step
- KV cache memory is precisely computable: `2 × L × n_kv_heads × d_k × seq_len × bytes_per_element`; GQA's reduction in KV heads is the primary architectural lever for reducing this cost
- Naive contiguous allocation wastes 60–80% of KV cache memory through internal and external fragmentation; PagedAttention eliminates both by allocating fixed-size pages on demand, achieving 90%+ utilization
- Prefix sharing lets multiple requests reference the same physical KV cache pages for a shared prompt prefix — identical K/V vectors computed once, stored once; radix attention generalizes this to arbitrary shared structure
- INT8 KV cache quantization provides a 2× memory reduction with negligible quality loss, effectively doubling serving concurrency on fixed hardware
- Eviction policies (sliding window, H2O, SnapKV) enable sequences longer than the cache capacity; attention sink preservation is mandatory — evicting the BOS token causes unpredictable quality degradation
- StreamingLLM enables unbounded generation on a fixed cache budget by combining BOS token preservation with a sliding recent window
- CPU offloading moves inactive request caches to DRAM, freeing GPU HBM for active requests at the cost of PCIe transfer latency on resumption
- KV cache metrics — utilization, prefix hit rate, eviction rate, throughput vs. sequence length — are the primary production signals for diagnosing memory pressure and serving efficiency

---

7.11 Further reading

- Kwon et al. (2023). *Efficient Memory Management for Large Language Model Serving with PagedAttention.* — the foundational work on paged KV cache management.
- Zheng et al. (2023). *SGLang: Efficient Execution of Structured Language Model Programs.* — Introduces radix attention for generalized prefix tree sharing.
- Xiao et al. (2023). *Efficient Streaming Language Models with Attention Sinks.* — StreamingLLM and the attention sink phenomenon; directly relevant to eviction strategy design.
- Zhang et al. (2023). *H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models.* — Attention-score-based eviction.

← Previous: 08 — Attention computation
Next: 10 — GPU execution →

Core Idea

- Autoregressive decoding recomputes attention over all previous tokens
- KV cache stores Keys & Values once and reuses them for all future steps
- Eliminates redundant projection + recomputation during generation

`Without cache: O(T²) recomputation → With cache: O(T)`

Why It Matters in Production

Each new token attends to all previous tokens in KV cache

Decode cost becomes memory-bandwidth bound (HBM), not compute bound

Long context → KV cache dominates GPU memory and throughput

What is Stored

For each layer:

`K: [seq_len, heads, d_k]`

`V: [seq_len, heads, d_v]`

Stored per token, per layer, per head

Q is NOT stored (used once per step)

Growth: linear in sequence length

Attention with KV Cache

`o_t = softmax(Q_t K_cache^T) V_cache`

At step t:

- Q_t computed fresh
- K/V appended once per token
- All past K/V reused

No recomputation of history

Without vs With KV Cache

Without cache:

`Recompute K,V for all tokens at every step → O(T²)`

With cache:

`Compute K,V once → reuse → O(T)`

Memory Cost

`KV bytes = 2 × L × n_kv_heads × d_k × seq_len × bytes`

Key drivers:

- Layers (L)
- KV heads (GQA reduces this)
- Context length (linear growth)

Decode Bottleneck

Each token requires reading full KV cache

→ HBM bandwidth bound (not FLOPs)

At long context:

`1–10 GB memory read per token step`

Result: longer context = slower generation

Prefill vs Decode

Prefill:

`Parallel matmul → compute-bound`

Decode:

`Vector × matrix → bandwidth-bound`

Same model, different bottleneck regime

Key Optimizations

- GQA: reduce KV heads → smaller cache
- PagedAttention: avoid fragmentation (paging instead of contiguous memory)
- Prefix sharing: reuse KV for shared prompts
- INT8/FP8 KV cache: 2× memory reduction
- Eviction policies: sliding window / H2O / SnapKV

`Goal: maximize GPU utilization under fixed HBM budget`

Figure 7.2: Cheat sheet.

Chapter 8

GPU Execution

The canonical question for this chapter: *When the model runs, what is GPU actually doing? Why does the same operation take wildly different amounts of time depending on how it is scheduled, and how the data is laid out in memory?*

Where are we?

Figure 8.1: The journey through the Model Mind.

The previous chapters established what the model computes: embeddings, attention, feed-forward networks, the KV cache. This chapter covers *where* that computation actually happens and why it behaves the way it does. Every optimization discussed so far (FlashAttention, PageAttention, continuous batching, quantization) exists because of specific GPU execution constraints. This chapter makes those constraints explicit.

8.1 Why GPU execution deserves its own chapter

Every previous chapter has mentioned GPU constraints in passing: memory bandwidth, arithmetic throughput, HBM capacity. This chapter makes those concepts precise.

Understanding GPU execution has direct impact ob system performance and it is not just an academic exercise for LLM engineers. A serving system operating at 60% of a GPU's theoretical peak throughput can deliver 50% more throughput than one running at just 40%, without any additional hardware. This knowledge also explains why certain operations become bottlenecks, why quantization improves performance, why kernel fusion is so effective, and why the prefill and decode phases show different latency characteristics.

More directly: every optimization technique in serving (FlashAttention, continuous batching, tensor parallelism) exists because of specific GPU execution constraints. Understanding those constraints explains why the optimizations take the form they do, rather than appearing as a collection of unrelated tricks.

8.2 GPU architecture fundamentals

A modern GPU is not a fast CPU. It is a fundamentally different compute architecture optimized for a different class of problems.

8.2.1 The execution model

A CPU has a small number of powerful cores (typically 8–128), each capable of complex out-of-order execution, branch prediction, and deep caches. CPUs are optimized for latency and completing a single thread of execution as fast as possible.

A GPU has a massive number of simpler cores (an H100 has 16,896 CUDA cores) organized into streaming multiprocessors (SMs). Each SM is not particularly fast for a single operation, but thousands of them executing simultaneously produce enormous total throughput. GPUs are optimized to complete as many simple operations as possible per second, not completing any one operation quickly.

The programming model reflects on this: GPU programs are written as kernels that execute the same operation on many data elements simultaneously. A matrix multiplication kernel runs the same multiply operation on thousands of elements in parallel. If there is data parallelism in your problem, a GPU exploits it. If there is not, a GPU is slower than a CPU for that specific task.

LLM inference has enormous data parallelism: the same transformer operations apply to every token in the sequence, and every element of every weight matrix participates in the same type of computation. This structural match is why GPUs dominate LLM inference.

8.2.2 The memory hierarchy

GPU memory has multiple tiers with very different bandwidth and capacity:

Memory tier	Bandwidth	Capacity	Latency
Registers	~80 TB/s	256 KB/SM	~1 cycle
L1/SRAM	~20 TB/s	256 KB/SM	~30 cycles
L2 cache	~5 TB/s	50 MB	~200 cycles
HBM (VRAM)	~3.35 TB/s	80 GB	~600 cycles
PCIe (CPU RAM)	~64 GB/s	TBs	~microseconds
NVLink (GPU-GPU)	~900 GB/s	-	~microseconds

SRAM (often called shared memory or L1 cache) is the GPU's fast on-chip memory. It offers extremely low latency but limited capacity. HBM (High Bandwidth Memory), is the GPU's large off-chip memory although it provides massive bandwidth, accessing HBM is still significantly more expensive than accessing on-chip memory.

Every GPU computation ultimately reads data from HBM and writes results back to it. As a result, the central challenge of GPU kernel optimization is reducing HBM traffic by keeping frequently accessed data in SRAM for as long as possible which is the exact key insight behind FlashAttention.

8.2.3 Streaming multiprocessors and warps

The Streaming Multiprocessor (SM) is the fundamental compute unit of an NVIDIA GPU. An H100 GPU contains many SMs operating in parallel, with each SM responsible for executing thousands of lightweight threads concurrently.

Each SM on an H100 contains:

- 128 CUDA cores for FP32 arithmetic operations.
- 4 Tensor Cores specialized for matrix multiplication.
- 256 KB of registers for storing temporary values used by active threads.
- 228 KB of on-chip SRAM, configurable between:
  - L1 cache for automatically caching memory accesses.

  - Shared memory for explicitly sharing data between threads in the same thread block.
- Warp schedulers that determine which groups of threads execute next.

GPU threads execute in groups called warps, where each warp consists of 32 threads. Unlike CPU threads, which execute independently, all threads within a warp execute the same instruction at the same time on different pieces of data. NVIDIA refers to this execution model as SIMT (Single Instruction, Multiple Threads).

For example, consider the following code executed by a warp:

```
if x > 0:
    y = a + b
else
    y = c + d
```

Suppose that among the 32 threads in the warp 20 threads satisfy $x > 0$ and 12 threads satisfy $x <= 0$ The GPU cannot execute both branches simultaneously within the same warp. Instead, it execute $y = a + b$ for the 20 relevant threads while the other 12 threads remain temporarily inactive and than execute $y = c + d$ for the remaining 12 threads while the first 20 threads remain inactive. Conceptually, execution looks like:

```
Cycle 1:
[Active]   Threads 0-19  -> y = a + b
[Inactive] Threads 20-31

Cycle 2:
[Inactive] Threads 0-19
[Active]   Threads 20-31 -> y = c + d
```

Although each thread only needs one branch, the warp executes both branches sequentially, reducing parallel efficiency whcih is known as branch divergence.

An H100 has 132 SMs. With 4 warp schedulers per SM and 32 threads per warp, the H100 can keep $132 \times 4 \times 32 = 16{,}896$ threads active simultaneously.

8.2.4 Tensor cores

Standard CUDA cores perform scalar floating point operations. Tensor cores perform matrix multiplications directly in hardware, at much higher throughput.

An H100 tensor core performs a 16×16×16 matrix multiplication in a single operation. The H100 has 528 tensor cores, delivering approximately:

- 1,000 TFLOPS for BF16/FP16 matrix multiplications
- 1,979 TFLOPS for FP8 matrix multiplications
- 66 TFLOPS for FP32 scalar operations

The gap between tensor core throughput and scalar throughput explains why matrix multiplication is fast and why all other operations are relatively expensive on modern GPUs.

8.3 The roofline model

The roofline model is a simple framework for understanding whether a given operation is compute-bound or memory-bandwidth-bound, and what the theoretical maximum performance is:

```
Attainable performance = min(Peak FLOPS, Memory bandwidth × Arithmetic intensity)
```

Where arithmetic intensity = FLOPs executed / bytes read from memory.

The ridge point is where the roofline transitions from memory-bound to compute-bound which is approximately 300 FLOPs/byte for the H100 (1,000 TFLOPS / 3.35 TB/s). Operations with arithmetic intensity above 300 are compute-bound and those below are memory-bandwidth-bound.

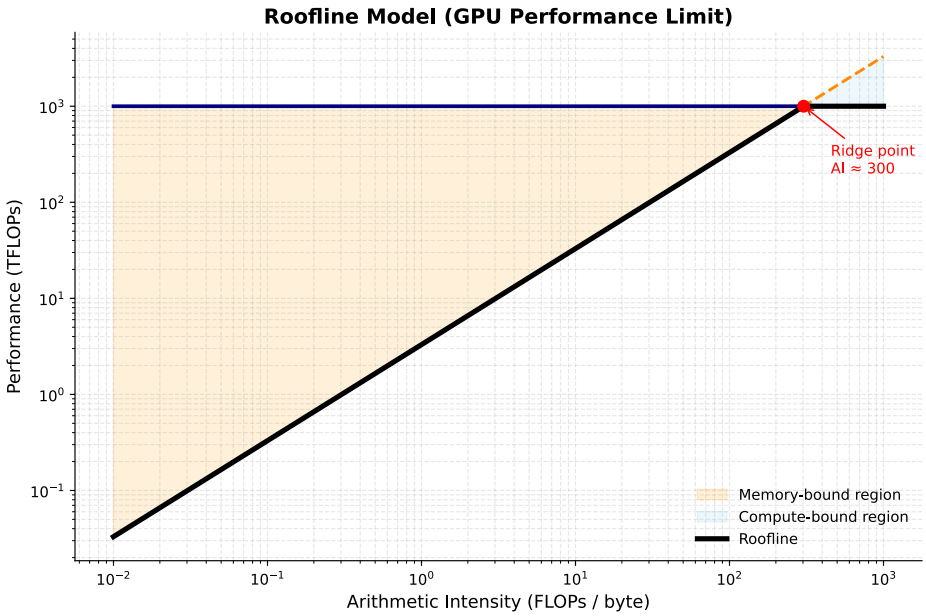


Figure 8.2: The roofline Model.

8.3.1 LLM operations on the roofline

Different operations in LLM inference have very different arithmetic intensities:

Large matrix multiplications (prefill, large batches): Matrix multiplication achieves high arithmetic intensity by reusing each byte of weight data for every sequence in the batch. For example, with a batch size of 32 and $d_model = 4,096$, the arithmetic intensity is approximately 32 FLOPs/byte. This value approaches the roofline model's ridge point, indicating that performance is primarily limited by computational throughput rather than memory bandwidth.

Vector-matrix products (decode, batch size 1): The Q projection at a batch size of 1 exhibits low arithmetic intensity because each byte of weight data is used in only one multiply operation. Consequently, the arithmetic intensity is approximately 1 FLOP/byte, which is about 300× below the roofline model's ridge point. As a result, performance is severely limited by memory bandwidth, leaving the tensor cores underutilized.

Attention with KV cache (decode): extremely low arithmetic intensity. Reading the full KV cache and computing a dot product with the query vector. Memory- bandwidth-bound with near-zero compute utilization.

Softmax, layer norm, element-wise ops: essentially zero arithmetic intensity. Read data, apply a simple function, write it back. Pure memory bandwidth operations. No tensor core involvement.

The practical implication: for decode at small batch sizes, the GPU is almost entirely idle arithmetically, waiting for data from HBM. Increasing batch size is the primary lever for improving GPU utilization during decode, because it amortizes weight reads across multiple sequences and increases arithmetic intensity toward the ridge point.

8.4 Prefill vs. decode: two different regimes

The roofline analysis explains why prefill and decode behave so differently and they are not just slow and fast versions of the same operation. They are different hardware bottlenecks entirely.

8.4.1 Prefill: compute-bound

During prefill, all prompt tokens are processed simultaneously. The Q projection operates on a matrix of shape `[n_prompt, d_model]`:

```
Q projection: [n_prompt, d_model] @ [d_model, d_model]
FLOPs:        2 × n_prompt × d_model²
Bytes:        d_model² × 2    (weight matrix, loaded once)

Arithmetic intensity = n_prompt FLOPs/byte
```

For n_prompt = 1,024 and d_model = 4,096: arithmetic intensity = 1,024 FLOPs/byte which is above the ridge point. Prefill is compute-bound. The H100 runs near its theoretical FLOPs ceiling. In order to make prefill faster it requires more FLOPs (more GPUs, higher clock speeds, or more efficient kernels).

8.4.2 Decode: memory-bandwidth-bound

During decode, only one new token is processed at a time. The Q projection operates on a vector of shape `[1, d_model]`:

```
Q projection: [1, d_model] @ [d_model, d_model]
FLOPs:        2 × 1 × d_model²
Bytes:        d_model² × 2    (same weight matrix)

Arithmetic intensity = 1 FLOPs/byte
```

For d_model = 4,096: arithmetic intensity = 1 FLOPs/byte which makes decode memory-bandwidth-bound. The H100 runs at approximately 1/300th of its peak FLOPs utilization. Making decode faster requires loading weights faster, higher bandwidth memory, quantization (fewer bytes per parameter), or larger batches (sharing weight reads across sequences).

8.5 Kernel fusion

Each operation in a transformer block, such as layer normalization, Q/K/V projection, softmax, and residual addition, can be implemented as a separate CUDA kernel. In this approach, every kernel reads its inputs from HBM and writes its outputs back to HBM before the next kernel executes. While this design is straightforward to implement, it is inefficient because intermediate results are repeatedly transferred to and from HBM despite being needed only by subsequent operations.

Kernel fusion addresses this inefficiency by combining multiple operations into a single CUDA kernel. Intermediate results are retained in registers or on-chip shared memory (SRAM) rather than being written to HBM after each operation. Only the final output is stored in HBM, reducing memory traffic, increasing arithmetic intensity, and improving overall performance.

8.5.1 Example: fused layer norm and linear projection

Without fusion:

```
x_norm = layer_norm(x)     # read x from HBM, write x_norm to HBM
q = x_norm @ W_Q           # read x_norm from HBM, write q to HBM
```

Two HBM reads, two HBM writes, for data that is only an intermediate result.

With fusion:

```
Single kernel:
  Read x from HBM once
```

```
Compute layer norm in registers/SRAM
Apply W_Q projection immediately
Write q to HBM once
```

One HBM read, one HBM write. Memory traffic halved for this pair of operations. The intermediate `x_norm` never leaves the chip.

8.5.2 FlashAttention as kernel fusion

FlashAttention is the most impactful example of kernel fusion in LLM inference. The naive attention computation reads and writes the n×n score matrix multiple times (scaling, masking, softmax, and the final weighted sum). FlashAttention fuses all of these into a single kernel that keeps intermediate scores in SRAM.

The memory traffic reduction is from $O(n^2)$ to $O(n)$. At long sequence lengths, this is the difference between attention fitting in available memory and not fitting at all. Chapter REFF covers FlashAttention in detail; the point here is that it is fundamentally a kernel fusion optimization, not an algorithmic approximation. The result is mathematically identical to naive attention.

8.5.3 Kernel fusion in practice

Production inference frameworks apply kernel fusion throughout the model:

- Layer norm fused into linear projections (Q, K, V, output projections)
- Attention scores, masking, and softmax fused (FlashAttention)
- Residual addition fused with layer norm
- Activation functions fused into FFN computation
- RoPE application fused into Q and K projection

The cumulative impact is typically 1.5–2× throughput improvement over naively separate kernels, primarily by reducing HBM traffic.

8.6 Precision and quantization at the kernel level

Quantization affects GPU execution at the kernel level, not just memory footprint. Understanding this distinction separates knowing what quantization does from knowing why it helps.

8.6.1 Weight-only quantization

The most common serving quantization scheme: model weights are stored in INT4 or INT8, but computation happens in BF16/FP16.

During a weight-only quantized matrix multiplication:

```
1. Load INT4 weights from HBM          (4× fewer bytes than BF16)
2. Dequantize to BF16 in registers     (fast, negligible compute overhead)
3. Multiply with BF16 activations via tensor cores
4. Accumulate in FP32, write BF16 output to HBM
```

The benefit: decode is memory-bandwidth-bound. Reducing weight bytes by 4× means 4× more weights can be streamed per second, directly improving decode throughput by approximately 4× for bandwidth-limited operations.

The cost: dequantization adds compute overhead. For bandwidth-bound operations (decode), this overhead is negligible. For compute-bound operations (prefill, large batches), dequantization adds meaningful overhead. Weight-only quantization therefore improves decode more than prefill, which is precisely where the improvement is most needed.

8.6.2 Activation quantization: INT8 and FP8

For compute-bound operations, quantizing both weights and activations to INT8 or FP8 allows tensor cores to operate at higher throughput:

- BF16 tensor cores: 1,000 TFLOPS
- FP8 tensor cores: 1,979 TFLOPS on H100 (approximately 2× faster)

Activation quantization is more complex than weight quantization because activations have more dynamic range and they can contain outlier values that cause large quantization errors when mapped to a narrow integer range.

The outlier problem. LLM activations contain outlier values in a small fraction of channels, values that are orders of magnitude larger than typical. These outliers are consistent across inputs and are a known property of large language models. Naive quantization sets the scale by the outlier, compressing all normal values to near-zero and destroying information.

LLM.int8() (Dettmers et al., 2022) addresses this by decomposing the matrix multiplication: outlier channels are computed in full BF16 precision, and the remaining channels (typically 99%+) are computed in INT8. The results are combined. Quality is preserved while most of the memory savings are retained.

SmoothQuant (Xiao et al., 2023) migrates the quantization difficulty from activations to weights, mathematically equivalent to scaling the activation channels down and the corresponding weight rows up before quantization. The result is more uniform activation distributions that quantize cleanly to INT8 without special outlier handling.

FP8 inference is available on H100 GPUs and has become the recommended precision for large-scale serving: near-BF16 quality at roughly 2× the arithmetic throughput. The H100's Transformer Engine handles FP8 scaling automatically, making it practical without manual calibration.

8.7 Tensor parallelism at the kernel level

When a model is too large for a single GPU, tensor parallelism distributes weight matrices across multiple GPUs. The kernel-level implementation requires communication between GPUs at each layer.

8.7.1 Column and row parallelism

A linear projection `Y = X @ W` can be split in two ways:

Column parallelism splits W by columns:

```
W split into [W  | W ] across 2 GPUs
GPU 0: Y  = X @ W     (partial output - subset of output dimensions)
GPU 1: Y  = X @ W     (partial output - complementary subset)
Combine: Y = [Y  | Y ]    (concatenation, no communication needed here)
```

Row parallelism splits W by rows and requires split input:

```
W split into [W ; W ], X split into [X , X ] across 2 GPUs
GPU 0: Y_partial  = X  @ W
GPU 1: Y_partial  = X  @ W
Combine: Y = Y_partial  + Y_partial     (all-reduce required)
```

In a transformer layer: Q, K, V projections use column parallelism, each GPU computes a subset of attention heads. The output projection uses row parallelism. Attention within each GPU is independent; an all-reduce synchronizes after the output projection. The FFN follows the same pattern: column parallelism on the first layer, row parallelism on the second, with one all-reduce between them.

8.7.2 Pipeline parallelism and bubble overhead

Pipeline parallelism splits a transformer model across multiple GPUs by assigning different sets of layers to different devices. Instead of each GPU holding the full model, computation flows through GPUs sequentially, like an assembly line and each GPU processes a contiguous block of layers. To keep all GPUs busy, the input batch is divided into microbatches, which are then streamed through the pipeline:

```
Time step:         1    2    3  |  4    5    6    7    8  | 9   10   11
                 <-- Fill -->  | <---- Steady state ---> | <-- Drain -->

GPU 0 (L0 -L24): [1]  [2]  [3]   [4]  [5]  [6]  [7]  [8]   ·    ·    ·
GPU 1 (L25-L48):  ·   [1]  [2]   [3]  [4]  [5]  [6]  [7]  [8]   ·    ·
GPU 2 (L49-L72):  ·    ·   [1]   [2]  [3]  [4]  [5]  [6]  [7]  [8]   ·
GPU 3 (L73-L96):  ·    ·    ·    [1]  [2]  [3]  [4]  [5]  [6]  [7]  [8]
```

This creates a wave-like execution pattern where different GPUs work on different microbatches at different stages of the model. However, pipeline parallelism introduces idle time, known as the pipeline bubble. This occurs during pipeline fill (startup) and pipeline drain (shutdown), when not all GPUs are fully utilized.

Increasing the number of microbatches reduces bubble overhead by improving pipeline utilization, but it also increases memory usage and scheduling complexity. In practice, systems use 8–32 microbatches to keep bubble overhead under 10% while maintaining efficiency.

8.8 CUDA graph capture

Every GPU operation requires a kernel launch: the CPU sends an instruction to the GPU specifying which kernel to run with which parameters. For small operations, kernel launch overhead can dominate the actual execution time.

For a transformer decode step at batch size 1:

- Total arithmetic work: small (bandwidth-bound, most of step time is HBM reads)
- Number of kernel launches: hundreds (one per operation across all layers)
- Kernel launch overhead: ~5–10 microseconds per launch

Hundreds of launches × 5–10 microseconds = milliseconds of CPU overhead per decode step, for a step that may itself take only a few milliseconds of actual GPU work. The CPU becomes the bottleneck.

CUDA graphs capture the sequence of kernel launches and their data dependencies as a static graph, replayed without CPU involvement for each step. After the initial capture, subsequent replays bypass CPU overhead entirely.

For decode at batch size 1, CUDA graph capture reduces per-step latency by 30–50%. The tradeoff: the graph is captured for a specific batch size and sequence length; changing these requires recapture. Production inference frameworks use CUDA graphs for decode (fixed batch size and one new token per step) and not for prefill (variable sequence length per request).

8.9 Key takeaways

- GPUs are throughput-optimized processors with thousands of simple cores; LLM inference maps well to GPUs because transformer operations are highly data- parallel — the same computation applied to every token and every weight element
- The GPU memory hierarchy spans registers (80 TB/s) to HBM (3.35 TB/s) to PCIe (64 GB/s); kernel optimization is the art of keeping hot data in fast memory and minimizing HBM round trips
- Tensor cores perform 16×16×16 matrix multiplications in hardware at ~15× the throughput of scalar CUDA cores; every operation expressible as matrix multiplication should be

- The roofline model predicts whether an operation is compute-bound (arithmetic intensity above ~300 FLOPs/byte for H100) or memory-bandwidth-bound (below); prefill is compute-bound, decode is severely memory-bandwidth-bound at ~1 FLOPs/byte
- MFU for decode at batch size 1 is approximately 0.3% — the GPU is nearly idle arithmetically; batching and quantization are the primary levers for improving this
- Kernel fusion eliminates intermediate HBM reads and writes between operations; FlashAttention reduces attention memory traffic from $O(n^2)$ to $O(n)$ by keeping scores in SRAM throughout computation
- Weight-only INT4/INT8 quantization reduces decode latency proportionally to the bit reduction, with negligible dequantization overhead when bandwidth-bound; FP8 activation quantization improves prefill throughput by ~2×
- CUDA graph capture eliminates CPU kernel launch overhead — 30–50% latency reduction for decode at small batch sizes — by replaying a static kernel sequence without CPU involvement

---

8.10 Further reading

- Williams et al. (2009). *Roofline: An Insightful Visual Performance Model for Multicore Architectures.* — The roofline model; essential reading for GPU performance analysis regardless of domain.
- Dao et al. (2022). *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness.* — The most important kernel fusion example in LLM inference; the IO complexity analysis is the key insight.
- Dettmers et al. (2022). *LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale.* — The outlier problem and mixed-precision INT8 quantization.
- Xiao et al. (2023). *SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models.* — Migrating quantization difficulty from activations to weights for clean INT8 inference.
- Shoeybi et al. (2019). *Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism.* — Tensor and pipeline parallelism; the foundational implementation reference.

← Previous: 09 — KV cache
Next: 11 — Decoding and sampling →

Transformer Architecture — Cheat Sheet

Why Transformers Won

- RNNs: sequential → no parallelism
- Long-range info bottleneck in hidden state
- Transformers: attention → full parallelism + direct token interaction

High-Level Pipeline

Tokens → Embeddings → Positional Encoding → Transformer Blocks → Logits → Softmax

`P(next token | context)`

Embeddings & Position

Token embedding: lookup table (vocab → vectors)

Weight tying: input embedding = output projection

Positional encodings:

- Sinusoidal (fixed, original transformer)
- Learned embeddings
- RoPE: rotates Q/K → relative position

Self-Attention

`Q = XWq, K = XWk, V = XWv`

`Attention = softmax(QKᵀ / √d) V`

Each token attends to all previous tokens

Creates contextualized representation

Attention Masks

- Causal (decoder): only past tokens visible
- Bidirectional (encoder): full sequence visible (BERT)
- Encoder-decoder: cross-attention between memory + generation
- Prefix LM: prefix bidirectional, suffix causal

Multi-Head Attention

Multiple parallel attention heads

Each head learns different relation type

Outputs concatenated + projected

`Specialization emerges from training`

Feed-Forward Network

`FFN(x) = W2 · activation(W1 x)`

Per-token independent transformation

Modern activations:

- GELU (default GPT/BERT)
- SwiGLU (modern LLaMA, Mistral)

Residual + Normalization

x = x + Sublayer(LayerNorm(x)) (Pre-LN)

Stabilizes deep training

LayerNorm → RMSNorm (modern models)

Pre-LN = stable gradients at scale

Modern Variants

- RoPE: relative position encoding
- GQA: shared K/V across heads
- MoE: sparse expert FFNs
- Sliding window attention (Mistral)

Output Layer

`logits = h Wᵀ → softmax → P(next token)`

Training objective: next-token prediction only

Figure 8.3: Cheat sheet.

Part IV

The Forbidden Archive

Part V

The Forging of Intelligence

Part VI

The Trial of the Answer

Part VII

The Keepers of the Waking Engine

[1] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A distributed serving system for {transformer-based} generative models," in *16th USENIX symposium on operating systems design and implementation (OSDI 22)*, 2022, pp. 521–538.